Theoretical Framework: crystal-prop-bench

Companion to: `crystal-prop-bench` implementation plan **Scope:** Mathematical foundations for materials property prediction with calibrated uncertainty and domain-shift evaluation **Target readers:** Hiring reviewers at Hereon (ML + UQ + XAI) and CuspAI (crystal-graph models + generation pipelines) **Status:** Phase A shipped. Phase B (ALIGNN) shipped as a single-seed benchmark integrated into the common evaluation pipeline; full 3-seed Tier 3 statistical parity with Tiers 1–2 is deferred due to GPU training cost (see DECISIONS.md topic 22). Phase A implementation references use past/present tense. Section 7 is written for the shipped ALIGNN architecture (Choudhary & DeCost, 2021). Section 11 reports the 11 measured findings from Phase A and Phase B.

Table of Contents

1. Motivation & Physical Setup
2. Feature Representation Theory
3. Supervised Learning: Gradient Boosting
4. Uncertainty Quantification: Split Conformal Prediction
5. Domain Shift Framework
6. Explainability: Shapley Values and TreeSHAP
7. Crystal Graph Neural Networks (Phase B)
8. Reference Paper Connections
9. Theory-to-Deliverable Mapping
10. Validation Criteria
11. Reported Findings
12. Notation Reference

---

1. Motivation & Physical Setup

1.1 What Physical System Are We Studying?

A **crystal** is a solid whose constituent atoms are arranged in a periodically repeating pattern. The minimal repeating unit is the **unit cell**, described by:

- **Lattice vectors** $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3 \in \mathbb{R}^3$ defining the periodicity
- **Basis atoms** at fractional coordinates $\mathbf{r}_j = u_j\mathbf{a}_1 + v_j\mathbf{a}_2 + w_j\mathbf{a}_3$ for $j = 1, \dots, N_{\text{atoms}}$
- **Chemical species** $Z_j \in \{1, \dots, 118\}$ (atomic number) for each basis atom

The **space group** G encodes the full symmetry of the crystal — the set of isometries (rotations, reflections, screw axes, glide planes) that map the structure onto itself. There are 230 crystallographic space groups in three dimensions. The space group constrains which physical properties are allowed: for example, piezoelectricity requires a non-centrosymmetric space group.

We study two scalar properties computed from **density functional theory (DFT)**:

Formation energy per atom. Let $E_{\text{DFT}}^{\text{total}}(\text{crystal})$ be the total DFT energy of the crystal, n_i the number of atoms of element i, μ_i the chemical potential of element i in its standard reference state, and $N_{\text{atoms}} = \sum_i n_i$ the total number of atoms in the unit cell. The formation energy per atom is:

$$E_f = \frac{1}{N_{\text{atoms}}} \left(E_{\text{DFT}}^{\text{total}}(\text{crystal}) - \sum_i n_i \mu_i \right) \tag{1.1}$$

measured in eV/atom. Formation energy quantifies thermodynamic stability *relative to decomposition into elemental reference states*: $E_f < 0$ indicates the compound is energetically favorable compared with its separated elemental constituents. *[Assumption: This is a weaker statement than thermodynamic stability in general. A compound with $E_f < 0$ may still be unstable against decomposition into competing compounds — that question is answered by the energy above the convex hull, which is a different quantity. We use E_f as the regression target because it is directly available in Materials Project and does not require hull construction.]*

Band gap. The band gap E_g (eV) is the energy difference between the highest occupied electronic state (valence band maximum) and the lowest unoccupied state (conduction band minimum):

$$E_g = E_{\text{CBM}} - E_{\text{VBM}} \geq 0 \tag{1.2}$$

$E_g = 0$ defines a metal; $E_g > 0$ defines a semiconductor or insulator. Band gap governs optical absorption, electrical conductivity, and photovoltaic efficiency. *[Assumption: We use the Kohn-Sham band gap from PBE-GGA calculations in Materials Project. PBE systematically underestimates band gaps by 30–50% relative to experiment (Perdew, Burke & Ernzerhof, 1996). This systematic bias is consistent across the dataset and does not invalidate relative comparisons, but absolute values should not be compared to experimental measurements without correction.]*

1.2 Why Does This System Matter?

AI-driven materials discovery aims to replace expensive DFT calculations (minutes to hours per crystal) with fast ML surrogates (milliseconds). In a **conditional crystal generation pipeline** — the kind operated by groups like CuspAI — the workflow is:

1. **Generator** produces candidate crystal structures conditioned on target properties
2. **Property predictor** (this project) evaluates whether candidates hit those targets
3. **Uncertainty quantifier** (this project) triages candidates: confident predictions → synthesis queue, uncertain → DFT validation queue
4. **Ranking function** orders thousands of candidates by predicted proximity to targets

The predictor is the bottleneck arbiter of the generation pipeline. If it is overconfident on out-of-distribution chemistries, the pipeline wastes synthesis resources on false positives. If its uncertainty estimates are miscalibrated, the triage step is useless. This project builds the predictor with calibrated UQ and honest domain-shift evaluation.

1.3 Why These Computational Methods?

We use **gradient-boosted decision trees** (LightGBM) on handcrafted features as the primary model family. This is a deliberate choice:

- Composition-only descriptors (Magpie) explain ~70–80% of formation energy variance in comparable inorganic crystal datasets (Ward et al., 2016 report $R^2 \approx 0.9$ on ICSD); we expect similar

performance on Materials Project. Reporting this strong baseline honestly is itself a finding — it sets the bar that any more complex model must beat.

- LightGBM trains in minutes on CPU, enabling rapid iteration with 3-seed variance estimates across multiple splits.
- Tree-based models admit exact Shapley value computation under the tree-path-dependent formulation (TreeSHAP; Lundberg et al., 2020), providing native explainability without post-hoc approximation.
- Split conformal prediction (§4) is model-agnostic and applies directly to LightGBM's point predictions.

Phase B adds **ALIGNN** (Atomistic Line Graph Neural Network; Choudhary & DeCost, 2021) as a structure-aware alternative that operates on a coupled atom-graph + line-graph representation. The line graph encodes bond-angle triplets explicitly, which provides the geometric deep-learning connection to CuspAI's equivariant architecture family (MACE, NequIP). The architectural choice and the pivot from the originally-planned vanilla CGCNN are documented in DECISIONS.md topic 17.

The dataset pipeline, featurizers, and chemistry-family classifier are implemented in `src/crystal_prop_bench/data/`*. Models are in* `src/crystal_prop_bench/models/`*. The evaluation framework is in* `src/crystal_prop_bench/evaluation/`*.*

---

2. Feature Representation Theory

The central question in materials ML is: **how do we represent a crystal as a fixed-length vector suitable for supervised learning?** We develop two tiers of representation, each grounded in a different physical assumption about what information determines properties.

2.1 Tier 1: Composition Descriptors (Magpie)

Physical assumption. *[Assumption]* Formation energy is primarily determined by the chemical composition — which elements are present and in what ratios — rather than by the geometric arrangement of atoms. This assumption is approximately valid for formation energy (Ward et al., 2016 report $R^2 \approx 0.9$ with composition-only models on ICSD) but weaker for band gap, which depends on electronic structure and hence geometry.

Construction. Given a composition $\mathrm{A}_{x_1}\mathrm{B}_{x_2}$ ... with element fractions $\{(Z_i, f_i)\}_{i=1}^{N_{\text{elem}}}$ where $\sum_i f_i = 1$, the **Magpie** descriptor set (Ward et al., 2016) computes summary statistics over tabulated elemental properties.

Let $p_k(Z)$ denote the k-th elemental property of element Z — for example, electronegativity, atomic radius, valence electron count, melting point, or any of $K \approx 22$ properties tabulated in the Magpie database. For each property p_k, we compute the **composition-weighted statistics**:

$$\mu_k = \sum_{i=1}^{N_{\text{elem}}} f_i \, p_k(Z_i) \tag{2.1}$$

$$\sigma_k^2 = \sum_{i=1}^{N_{\text{elem}}} f_i \left(p_k(Z_i) - \mu_k\right)^2 \tag{2.2}$$

$$\text{range}_k = \max_i \, p_k(Z_i) - \min_i \, p_k(Z_i) \tag{2.3}$$

along with min, max, and mode. This yields approximately $6K \approx 132$ features from composition statistics alone. Additional features include:

- **Stoichiometric attributes:** number of elements N_{elem}, L^p-norms of the fraction vector $\mathbf{f}$
- **Valence orbital fractions:** fraction of valence electrons in s, p, d, f orbitals
- **Ionic character indicators:** whether the compound can form a charge-balanced ionic composition

The total Magpie descriptor vector has dimensionality $d_1 \approx 150$.

Why this works. Formation energy reflects the energetics of chemical bonds. Chemical bonding is governed by electronegativity differences, orbital compatibility, and atomic size ratios — all of which are captured by composition statistics. The Magpie descriptors encode a physics-informed prior: instead of learning element-property relationships from scratch, the model receives tabulated physical constants as input and only needs to learn how their *combinations* relate to target properties.

Limitation. *[Exact]* Composition descriptors are *invariant to permutations of atom positions* — by construction, they cannot distinguish polymorphs (same composition, different crystal structure). Diamond and graphite have identical Magpie descriptors but dramatically different properties. For properties that depend on geometry (band gap, elastic moduli), this is a fundamental information bottleneck.

Implemented in `src/crystal_prop_bench/data/featurizers.py` *via* `matminer.featurizers.composition`
Features are cached to parquet after first computation.

2.2 Tier 2: Structure-Aware Tabular Descriptors

Physical assumption. *[Assumption]* Adding geometric information — density, symmetry, and local coordination environment — captures structure-property relationships that composition alone misses, particularly for band gap.

We augment the Magpie vector with three classes of structural descriptors:

2.2.1 Bulk descriptors.

- **Density** $\rho = \sum_i m_i / V_{\text{cell}}$ (g/cm³)
- **Volume per atom** $V_{\text{atom}} = V_{\text{cell}} / N_{\text{atoms}}$ (Å³/atom)
- **Packing fraction** $\eta = \sum_i (4\pi r_i^3/3)/V_{\text{cell}}$, where r_i is the Shannon ionic radius of the i-th atom

These are scalar summaries of the unit cell geometry. Packing fraction in particular correlates with structural type (close-packed metals vs. open-framework oxides).

2.2.2 Symmetry descriptors.

- **Space group number** $g \in \{1, \dots, 230\}$, encoded as an **ordinal integer feature**
- **Crystal system** $c \in \{$triclinic, monoclinic, orthorhombic, tetragonal, trigonal, hexagonal, cubic$\}$, encoded as ordinal (1–7)

[Assumption: We use ordinal encoding for both space group number and crystal system. This is the standard choice for tree-based models, which can split on any threshold and therefore extract non-linear relationships from ordinal features without requiring one-hot encoding. For space group number, the conventional index has some structural meaning (higher numbers tend toward higher symmetry). For crystal system, the 1–7 ordering is purely a coding convenience with no physically meaningful scalar interpretation — the tree treats it as an arbitrary ordered index and learns useful split points regardless. The encoding choice is documented in DECISIONS.md and affects SHAP interpretation: SHAP values for ordinal features reflect the contribution of being above or below a split threshold, not membership in a specific category.]

2.2.3 Local coordination environment via Voronoi tessellation.

This is the most physically meaningful structural descriptor class. For each atom i in the unit cell, we compute the **Voronoi cell** — the region of space closer to atom i than to any other atom (including periodic images).

Definition. The Voronoi cell of atom i at position $\mathbf{r}_i$ is:

$$\mathcal{V}_i = \left\{ \mathbf{r} \in \mathbb{R}^3 \,:\, |\mathbf{r} - \mathbf{r}_i| \leq |\mathbf{r} - \mathbf{r}_j| \;\; \forall j \neq i \right\} \tag{2.4}$$

where the $\mathbf{r}_j$ include all periodic images within a sufficient supercell. Two atoms i and j are **Voronoi neighbors** if their Voronoi cells share a face. The **solid angle** Ω_{ij} subtended by the shared face provides a continuous measure of neighbor weight — atoms with larger shared faces are "more neighboring."

Effective coordination number. We use matminer's `VoronoiNN` coordination number, which computes an effective coordination number by weighting neighbors by their Voronoi face solid-angle fraction:

$$\mathrm{CN}_i^{\mathrm{eff}} = \sum_{j \in \mathcal{N}(i)} w_{ij} \tag{2.5}$$

where w_{ij} is the solid-angle weight assigned by matminer's `VoronoiNN` implementation. *[Assumption: We adopt the default `VoronoiNN` weight convention in matminer. The exact weighting scheme — including face-area cutoff and normalization — is determined by the library defaults and is pinned in `pyproject.toml` via a minimum version constraint. The specific kwargs passed to `VoronoiNN` are documented in `DECISIONS.md`.]*

Why Voronoi over fixed-radius cutoffs. *[Assumption: Voronoi tessellation is the appropriate neighbor-finding algorithm for this project.]* A fixed-radius cutoff r_c requires choosing a single distance threshold — too small misses neighbors in open structures, too large includes non-bonded atoms in dense structures. The optimal r_c varies by chemistry family (e.g., ~3.5 Å for close-packed metals vs. ~5.0 Å for framework silicates). Voronoi tessellation is **parameter-free**: it determines neighbors from geometry alone, adapting automatically to the local environment. The tradeoff is computational cost (~10× slower than fixed-radius) and occasional failure on pathological geometries.

Site statistics. From the per-atom coordination numbers $\{\mathrm{CN}_i^{\mathrm{eff}}\}_{i=1}^{N_{\mathrm{atoms}}}$, we compute summary statistics: mean, std, min, max, range. Combined with the Magpie composition descriptors and bulk/symmetry features, the Tier 2 descriptor vector has dimensionality $d_2 \approx 200$.

Handling featurization failures. *[Exact]* Some crystal structures cause Voronoi tessellation to fail — typically structures with overlapping atomic sites, fractional occupancies, or highly distorted cells. These

are logged and dropped. The fraction dropped per chemistry family is reported as a diagnostic: if one family has a significantly higher failure rate, that indicates systematic structural differences relevant to downstream model performance.

2.2.4 Oxide subsampling for Tier 2 (DECISIONS.md topic 15).

Voronoi tessellation plus `SiteStatsFingerprint(VoronoiNN)` is computationally expensive — approximately 3 seconds per structure on a single core. On the full classified corpus of ~110K crystals, serial featurization would take ~80 hours; even with 8-core parallelism, the full set takes ~10 hours. To keep Phase A iteration loops short enough to support the 3-seed × 2-target × 2-split experimental matrix (12 Tier 2 runs), we apply a **stratified oxide subsample**: oxides are reduced from ~77K to 25K (stratified random, seed=42), while all minority families (sulfide, nitride, halide) are kept intact. The resulting Tier 2 dataset contains approximately 58K structures.

[Assumption: LightGBM's error curve plateaus well before 25K training examples for tabular features of dimensionality $d_2 \approx 200$. This is an empirical claim — gradient-boosted trees are generally data-efficient on tabular data, and the marginal gain from increasing oxide training size beyond 25K is small. The assumption is not verified by an explicit learning-curve study in this project; it is treated as a design decision with a documented fallback path (increase the subsample if the bias check in §5 reveals selection effects).]

Asymmetry between tiers. The subsample applies only to Tier 2. Tier 1 (Magpie) is evaluated on **two datasets**: - `tier1_full`: the full classified corpus (~110K crystals), which never fails composition featurization - `tier1_voronoi_subset`: the ~58K subset that survived Voronoi featurization and forms the Tier 2 training pool

Both Tier 1 evaluations appear as separate rows in `results/tables/benchmark.csv`. This is the **featurization bias check** — see §5.6.

Why minority families are never subsampled. The minority families (sulfide, nitride, halide) are the OOD test sets for the domain-shift protocol. Subsampling them would reduce the statistical power of the domain-shift evaluation and would contaminate the cross-tier comparison (Tier 1 sees the full minority distribution, Tier 2 sees a downsampled version). Keeping them intact preserves the clean tier comparison at the cost of slight computational asymmetry.

Implemented in `src/crystal_prop_bench/data/featurizers.py` via `matminer.featurizers.structure.S` with `VoronoiNN`-based `CoordinationNumber`. Featurization is parallelized via `featurize_dataframe(n_jobs=8)`. Features are cached to parquet.

---

3. Supervised Learning: Gradient Boosting

3.1 Problem Setup

We frame materials property prediction as a regression problem. Given a training set $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ where $\mathbf{x}_i \in \mathbb{R}^d$ is the feature vector (Tier 1 or Tier 2) and $y_i \in \mathbb{R}$ is the target property (formation energy per atom or band gap), we seek a function $\hat{f} : \mathbb{R}^d \to \mathbb{R}$ minimizing the expected loss:

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \mathbb{E}_{(\mathbf{x},y)\sim P} \left[L(y, f(\mathbf{x})) \right] \tag{3.1}$$

Training objective. LightGBM's `"objective": "regression"` minimizes the **squared-error** (L2) loss:

$$L_{\text{train}}(y, f(\mathbf{x})) = (y - f(\mathbf{x}))^2 \tag{3.2}$$

Primary evaluation metric. We report **mean absolute error** (MAE) as the primary metric, consistent with the materials ML literature:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{f}(\mathbf{x}_i)| \tag{3.3}$$

This distinction — L2 training loss, L1 evaluation metric — is standard. LightGBM's `"metric": "mae"` controls early stopping based on validation MAE, but the gradient updates use L2 residuals. See §8.7 (Friedman, 2001) for the gradient boosting derivation.

3.2 Gradient Boosting Framework

Gradient boosting (Friedman, 2001) constructs $\hat{f}$ as an additive ensemble of weak learners (decision trees):

$$\hat{f}(\mathbf{x}) = f_0 + \sum_{m=1}^{M} \nu \, h_m(\mathbf{x}) \tag{3.4}$$

where f_0 is the initial prediction (typically the training mean $\bar{y}$), h_m is the m-th decision tree, $\nu \in (0, 1]$ is the learning rate (shrinkage), and M is the number of boosting rounds.

Each tree h_m is fit to the **negative gradient** of the loss evaluated at the current ensemble prediction. For squared-error loss:

$$-\frac{\partial L(y_i, F)}{\partial F}\bigg|_{F=\hat{f}_{m-1}(\mathbf{x}_i)} = y_i - \hat{f}_{m-1}(\mathbf{x}_i) \equiv r_i^{(m)} \tag{3.5}$$

which is simply the residual. Tree h_m is trained to predict these residuals, and the ensemble is updated:

$$\hat{f}_m(\mathbf{x}) = \hat{f}_{m-1}(\mathbf{x}) + \nu \, h_m(\mathbf{x}) \tag{3.6}$$

LightGBM implementation. LightGBM (Ke et al., 2017) provides several optimizations over classical gradient boosting, including **Exclusive Feature Bundling** (EFB), which identifies mutually exclusive features (features that rarely take nonzero values simultaneously) and bundles them into a single feature, reducing the effective dimensionality. EFB is active by default and may be relevant given the sparse structure of some Magpie features.

LightGBM also supports **Gradient-based One-Side Sampling** (GOSS), which retains all instances with large gradients and randomly samples instances with small gradients. *[Note: GOSS is not enabled in the Phase A configuration, which uses the default `boosting_type="gbdt"` (standard gradient boosting).*

GOSS would require explicitly setting `boosting_type="goss"`. We note it here as a capability that could be explored in future optimization, but it is not part of the current method.]

Regularization. We control overfitting through:

- `num_leaves = 127`: Maximum number of leaves per tree. Controls model complexity — higher allows more complex split patterns but risks overfitting.
- `min_child_samples = 20`: Minimum samples per leaf, preventing splits on tiny subsets.
- `subsample = 0.8`: Row subsampling per tree (stochastic gradient boosting; Friedman, 2002).
- `colsample_bytree = 0.8`: Feature subsampling per tree, reducing feature correlation between trees.
- `early_stopping_rounds = 50`: Training stops if validation MAE does not improve for 50 consecutive rounds.

Training configuration. The Phase A experimental matrix is 3 seeds × 2 targets × 2 tiers × 2 splits (standard + domain-shift) = **24 LightGBM training runs**. Each run takes minutes on CPU. The 3-seed protocol follows the portfolio convention (also used in demandops-lite and finetune-bench) and provides sufficient variance estimates at this data scale. A mixed-training run (all chemistry families pooled) adds a further small set of runs for the domain randomization comparison in §5.5. Phase B (ALIGNN, §7) adds a separate set of single-seed runs on Modal A100 (DECISIONS.md topic 22).

Implemented in `src/crystal_prop_bench/models/lgbm_baseline.py`. Training scripts: `scripts/run_tier1.py`, `scripts/run_tier2.py`.

---

4. Uncertainty Quantification: Split Conformal Prediction

This section derives the UQ methodology from first principles. It is the theoretical core of the project.

4.1 The Calibration Problem

A regression model $\hat{f}$ produces point predictions $\hat{y} = \hat{f}(\mathbf{x})$. For deployment in a materials discovery pipeline, we need **prediction intervals** $C(\mathbf{x}) = [\hat{y} - q,\ \hat{y} + q]$ with a *guaranteed* coverage rate:

$$P\left(Y \in C(X)\right) \geq 1 - \alpha \tag{4.1}$$

where $\alpha \in (0, 1)$ is the user-specified miscoverage level (e.g., $\alpha = 0.10$ for 90% coverage). Classical approaches to this problem — bootstrap confidence intervals, Bayesian credible intervals, ensemble variance — require distributional assumptions that may not hold and do not provide finite-sample coverage guarantees.

4.2 Conformal Prediction: Setup

Split conformal prediction (Vovk et al., 2005; Lei et al., 2018) provides distribution-free prediction intervals with **exact finite-sample marginal coverage**, requiring only a single assumption:

[Assumption] Exchangeability. The calibration samples $(X_1, Y_1), \ldots, (X_n, Y_n)$ and the test point (X_{n+1}, Y_{n+1}) are exchangeable — meaning their joint distribution is invariant to permutations. This is strictly weaker than the i.i.d. assumption: exchangeability holds for i.i.d. data but also for certain

dependent sequences. Crucially, exchangeability is **violated under domain shift** — this is the theoretical basis for the expected behavior under the chemistry-family shift protocol (§5).

4.3 Derivation of Split Conformal Regression

Data splitting. Partition the data into three disjoint sets:

- $\mathcal{D}_{\text{train}}$: used to train $\hat{f}$
- $\mathcal{D}_{\text{cal}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$: used to calibrate the conformal interval
- $\mathcal{D}_{\text{test}}$: used for final evaluation

The model $\hat{f}$ is trained on $\mathcal{D}_{\text{train}}$ and then frozen. All subsequent steps use $\hat{f}$ only for inference.

Step 1: Compute conformity scores. Define the **nonconformity score** (residual) for each calibration point:

$$R_i = |Y_i - \hat{f}(X_i)|, \quad i = 1, \dots, n \tag{4.2}$$

These are the absolute residuals on the calibration set. Other score functions are possible (e.g., normalized residuals $R_i/\hat{\sigma}(X_i)$ for heteroscedastic models), but absolute residuals are the simplest and most interpretable choice for the MVP.

Step 2: Compute the conformal quantile. Sort the calibration residuals:

$$R_{(1)} \leq R_{(2)} \leq \cdots \leq R_{(n)} \tag{4.3}$$

For a desired coverage level $1 - \alpha$, compute the index:

$$k = \lceil (n+1)(1-\alpha) \rceil \tag{4.4}$$

The conformal quantile is then defined as:

$$\hat{q} = \begin{cases} R_{(k)} & \text{if } k \leq n \\ +\infty & \text{if } k > n \end{cases} \tag{4.5}$$

The case $k > n$ occurs when $\alpha < 1/(n+1)$, i.e., the requested coverage is so high that the calibration set is too small to support it. In that case, the conformal interval is infinite (conservative but formally correct).

[Implementation note: For the α values used in this project ($\alpha \in \{0.10, 0.20, 0.30\}$) with calibration sets of thousands, $k \leq n$ always holds, so the finite order-statistic branch is the only operational branch. In this regime, the quantile is computed as `np.quantile(residuals, q_level, method="higher")` *where* `q_level = np.ceil((n + 1) * (1 - alpha)) / n`*. The* `method="higher"` *interpolation is essential — it selects the smallest order statistic at or above the requested quantile, which is required for the finite-sample guarantee. Other interpolation methods (e.g.,* `"linear"`*, the NumPy default) may violate the coverage bound. The $k > n$ branch is included for theoretical completeness; if triggered in practice (e.g., by a very small calibration set or very small α), the implementation will explicitly return an infinite interval rather than clipping to $R_{(n)}$.]*

[Note on ties: If the model produces identical predictions for multiple calibration points (possible with tree models), the residuals may contain exact ties. The lower-bound coverage guarantee ($\geq 1-\alpha$) remains valid regardless of ties. However, with many ties, the actual coverage may exceed $1-\alpha+1/(n+1)$ — the interval becomes more conservative than the tie-free bound suggests. This is safe (over-coverage, not under-coverage) but means the upper bound in (4.7) is tight only in the tie-free case.]

Step 3: Construct the prediction interval. For a new test point $\mathbf{x}_{n+1}$:

$$C(\mathbf{x}_{n+1}) = \left[\hat{f}(\mathbf{x}_{n+1}) - \hat{q},\ \ \hat{f}(\mathbf{x}_{n+1}) + \hat{q}\right] \tag{4.6}$$

4.4 Coverage Guarantee

Theorem (Marginal Coverage). *[Exact]* If $(X_1, Y_1), \dots, (X_n, Y_n), (X_{n+1}, Y_{n+1})$ are exchangeable, then:

$$1-\alpha \leq P\left(Y_{n+1} \in C(X_{n+1})\right) \leq 1-\alpha+\frac{1}{n+1} \tag{4.7}$$

Proof. By exchangeability, the residuals $R_1, \dots, R_n, R_{n+1}$ are exchangeable. In the absence of ties, the rank of R_{n+1} among $\{R_1, \dots, R_{n+1}\}$ is uniformly distributed on $\{1, \dots, n+1\}$. We have $Y_{n+1} \in C(X_{n+1})$ iff $R_{n+1} \leq \hat{q} = R_{(k)}$, which occurs iff $\text{rank}(R_{n+1}) \leq k = \lceil (n+1)(1-\alpha) \rceil$. The probability of this event is $k/(n+1)$. Since $\lceil x \rceil \in [x, x+1)$, we get $k/(n+1) \in [1-\alpha,\ 1-\alpha+1/(n+1))$. The lower bound $1-\alpha$ remains valid in the presence of ties; however, with ties, the achieved coverage may exceed the upper bound $1-\alpha+1/(n+1)$, i.e., the interval may be more conservative than the tie-free analysis suggests. □

The upper bound shows the coverage guarantee is **tight** — we do not drastically overcover. For $n = 10{,}000$ (typical calibration set size), the excess coverage is at most 10^{-4}, which is negligible.

4.5 Marginal vs. Conditional Coverage

[Critical distinction.] The guarantee in (4.7) is **marginal** — it holds as an average over the test distribution $P(X)$. It does **not** guarantee conditional coverage:

$$P\left(Y \in C(X) \mid X \in \mathcal{A}\right) \geq 1-\alpha \tag{4.8}$$

for arbitrary subsets $\mathcal{A}$ of the input space. In fact, Barber et al. (2021) and Vovk (2012) show that conditional coverage is **impossible** to achieve in a distribution-free setting without further assumptions.

Why this matters for crystal-prop-bench. When the calibration set consists of oxides and the test set consists of a different chemistry family (domain-shift split), the exchangeability assumption is violated. The marginal coverage guarantee (4.7) no longer holds. **This prediction was confirmed in Phase A with concrete numbers** (§11, Finding 4). At $\alpha = 0.10$ (nominal 90% coverage):

Target	ID (oxide)	Sulfide	Nitride	Halide
Formation energy	90.2%	51.4%	24.7%	28.2%
Band gap	90.6%	89.3%	82.8%	60.0%

ID coverage matches the theorem within the $1/(n+1)$ bound (§4.4). OOD coverage collapses, exactly as predicted when exchangeability fails. A subtle additional result is that **band-gap coverage degrades less sharply than formation-energy coverage.** The mechanism: the band-gap model's residuals are heavier-tailed in-distribution, so $\hat{q}$ is already wide, and a wide interval absorbs more of the OOD distributional drift before coverage collapses. This is a practically important observation — a model with *worse* ID accuracy can have *more robust* conformal coverage under shift.

The theoretical vulnerability exposed here — marginal guarantees do not imply conditional safety — is not a bug in conformal prediction but a limitation of the exchangeability assumption under realistic deployment conditions. Section 4.7 derives the per-family recalibration recipe that addresses it.

4.6 Interval Width and the Informativeness-Coverage Tradeoff

The conformal interval in (4.6) has **constant width** $2\hat{q}$ for all test points — regardless of whether the model is confident or uncertain about a particular prediction. This is a known limitation of split conformal regression with absolute residuals as scores.

The tradeoff is controlled by α:

- Small α (high coverage, e.g., 90%): large $\hat{q}$, wide intervals, conservative but uninformative
- Large α (low coverage, e.g., 70%): small $\hat{q}$, tight intervals, informative but risky

We report at $\alpha \in \{0.10, 0.20, 0.30\}$ to trace this tradeoff. The **mean interval width** at each α quantifies informativeness. If one tier produces tighter intervals than another at the same coverage level, that tier is more useful for deployment triage.

Bonus C (CQR) addresses the constant-width limitation by training a quantile regression model to produce heteroscedastic intervals before conformalizing them, but this is deferred to post-MVP.

Split conformal regression is implemented in `src/crystal_prop_bench/evaluation/conformal.py`*. The function* `conformal_regression_interval()` *implements Equations (4.2)–(4.6) using the exact order-statistic convention specified above. Coverage analysis per family is in* `src/crystal_prop_bench/evaluation/domain_shift.py`*.*

4.7 Per-Family Recalibration Under Shift

Section 4.5 showed that exchangeability fails globally when the calibration set consists of oxides and the test set consists of a different chemistry family. The §4.4 marginal guarantee no longer applies in that setting. But the same theorem can be **reapplied per family**: if we obtain a small calibration sample drawn from family k, we can construct a new conformal interval that satisfies the marginal guarantee *conditional on being in family* k.

Setup. Let family k have its own joint distribution $P_k(X, Y)$. Suppose we have:

- A trained model $\hat{f}$ (trained on oxides — irrelevant to the guarantee, as long as $\hat{f}$ is frozen before we see the recalibration set)
- A **recalibration set** $\mathcal{D}_{\text{cal}}^{(k)} = \{(X_i^{(k)}, Y_i^{(k)})\}_{i=1}^{n_{\text{cal}}}$ with n_{cal} points drawn from P_k
- A test point $(X_{n_{\text{cal}}+1}^{(k)}, Y_{n_{\text{cal}}+1}^{(k)})$ also drawn from P_k

Then $\mathcal{D}_{\text{cal}}^{(k)}$ and the test point are exchangeable *within family* k by construction. Applying split conformal to these $n_{\text{cal}} + 1$ points using the family-k residuals $R_i^{(k)} = |Y_i^{(k)} - \hat{f}(X_i^{(k)})|$ yields a new quantile:

$$\hat{q}^{(k)} = R^{(k)}_{(\lceil (n_{\text{cal}}+1)(1-\alpha) \rceil)} \tag{4.9}$$

and a per-family prediction interval

$$C^{(k)}(\mathbf{x}) = \left[\hat{f}(\mathbf{x}) - \hat{q}^{(k)},\ \hat{f}(\mathbf{x}) + \hat{q}^{(k)}\right] \tag{4.10}$$

Theorem (Per-family coverage). *[Exact]* Under the conditions above, for any test point drawn from P_k:

$$1 - \alpha \leq P_k\left(Y \in C^{(k)}(X)\right) \leq 1 - \alpha + \frac{1}{n_{\text{cal}} + 1} \tag{4.11}$$

Proof. Identical to the proof of (4.7), replacing the global exchangeability assumption with within-family exchangeability. The per-family calibration set and the per-family test point are exchangeable by assumption (both drawn from P_k), so the rank argument in §4.4 applies verbatim within family k. □

What this theorem does and does not say. The guarantee is *marginal within family* k, not marginal over the full mixture of families. It does not claim global conditional coverage (which is still impossible by Barber et al., 2021). What it says is operationally more useful: if a practitioner encounters a new chemistry family and can afford a small number of DFT calculations to build a family-specific calibration set, they recover a valid coverage guarantee *on that family*, even though the globally-calibrated interval had failed.

How small can n_{cal} be? The theorem gives a finite-sample guarantee for any $n_{\text{cal}} \geq 1$, but the *variance* of $\hat{q}^{(k)}$ across different random draws of the calibration set grows as n_{cal} shrinks. In the limit $n_{\text{cal}} = 1$, the interval width is determined by a single residual and has enormous variance. The practical question — *how small can n_{cal} be before quantile variance dominates?* — cannot be answered by the theorem alone. It is an empirical question that the **calibration sweep experiment** (§4.8 mode 3) is designed to probe. The answer reported in Phase A is $n_{\text{cal}} \approx 25$ (see §11, Finding 6).

Connection to the literature. Per-family recalibration is a special case of the shifted-exchangeability recipe in Tibshirani et al. (2019), "Conformal Prediction Under Covariate Shift" (NeurIPS 2019). Their general framework handles arbitrary test-time distribution shifts via importance-weighted conformal prediction. Our setting is simpler: we *partition* the test distribution into families and apply standard split conformal within each family. This avoids the need to estimate importance weights but requires enough test-family samples to form a calibration set.

4.8 Three Conformal Evaluation Modes

The repository implements three conformal evaluation modes, each with a different theoretical status:

Mode 1 — Standard (in-distribution baseline). Calibrate on oxide calibration set, test on oxide test set. Exchangeability holds. The §4.4 marginal guarantee applies. **Measured in Phase A:** empirical coverage at $\alpha = 0.10$ was 90.2% (formation energy) and 90.6% (band gap), matching the theorem within the $1/(n+1)$ bound (§11, Finding 4).

Mode 2 — Cross-domain (failure demonstration). Calibrate on oxide calibration set, test on OOD family. Exchangeability is violated (training and test distributions differ systematically). The marginal guarantee no longer holds. **Measured in Phase A:** coverage collapses on OOD families, most severely for formation

energy (down to 24.7% on nitrides), less severely for band gap (down to 60.0% on halides). This mode is the empirical demonstration of §4.5.

Mode 3 — Calibration sweep (recovery). For each OOD family k, calibrate on $n_{\text{cal}} \in \{5, 10, 25, 50, 100\}$ samples drawn from k (repeated over multiple random draws to estimate variance), test on the remaining family-k samples. Within-family exchangeability holds by construction. The §4.7 per-family theorem applies. **Measured in Phase A:** $n_{\text{cal}} = 5$ is unreliable (std up to ±31%), $n_{\text{cal}} = 10$ recovers >90% for most families, $n_{\text{cal}} = 25$ restores coverage to 95–99% across all families and both targets, and 50–100 samples settle to the nominal 90% level as the small-n quantile conservatism disappears. This is the repo's headline deployable finding (§11, Finding 6).

All three modes are implemented in `src/crystal_prop_bench/evaluation/conformal.py`. Mode 3 uses `src/crystal_prop_bench/data/splits.py::ood_calibration_sweep()` to generate the per-family calibration-test partitions for each n_{cal} value.

---

5. Domain Shift Framework

5.1 Chemistry-Family Partitioning

We partition crystals into chemistry families by **dominant anion**. Let $\mathcal{A}(s)$ denote the set of anion sites in structure s, determined by oxidation-state assignment via pymatgen's `BVAnalyzer` (bond-valence method).

Failure policy for oxidation-state assignment. *[Assumption]* If `BVAnalyzer` raises an exception or returns an ambiguous assignment (e.g., no clear anion/cation distinction), the compound is dropped from the dataset and counted in the failure log. The fraction of dropped compounds due to oxidation-state assignment failure is reported alongside Voronoi featurization failures as a data-quality diagnostic. This rule is documented in `DECISIONS.md`.

Family definition. Define the family set:

$$\mathcal{H}_{\text{oxide}} = \{\text{O}\}, \quad \mathcal{H}_{\text{sulfide}} = \{\text{S}\}, \quad \mathcal{H}_{\text{nitride}} = \{\text{N}\}, \quad \mathcal{H}_{\text{halide}} = \{\text{F}, \text{Cl}, \text{Br}, \text{I}\} \tag{5.1}$$

For a structure s with anion sites $\mathcal{A}(s)$, define the **family fraction** for family k:

$$\phi_k(s) = \frac{|\{j \in \mathcal{A}(s) : Z_j \in \mathcal{H}_k\}|}{|\mathcal{A}(s)|} \tag{5.2}$$

A crystal is assigned to family $\mathcal{F}_k$ iff:

$$s \in \mathcal{F}_k \iff \phi_k(s) \geq \tau \tag{5.3}$$

where $\tau = 0.80$ is the **purity threshold**. Crystals that do not meet the purity threshold for any family are dropped. Since the family sets $\mathcal{H}_k$ are pairwise disjoint, the fractions $\phi_k(s)$ partition over disjoint subsets of anion sites, so at most one family can exceed the threshold $\tau = 0.80$ (because $2 \times 0.80 = 1.60 > 1.0$).

[Note: The halide family pools four element species into a single set $\mathcal{H}_{\text{halide}}$. A crystal with 50% F and 40% Cl anion sites has $\phi_{\text{halide}} = 0.90 \geq \tau$ and is classified as halide. This pooling decision reflects the chemical similarity of halogen anions and is documented in DECISIONS.md (topic 3).]

The number of dropped compounds (due to purity threshold, oxidation-state failure, or both) and their distribution are reported as diagnostics.

5.2 Covariate Shift as Working Assumption

Let P_S and P_T denote the source (training) and target (test) distributions over features and labels. The domain-shift protocol is designed to probe a regime **approximated** as covariate shift:

$$P_S(Y|X) \approx P_T(Y|X), \quad P_S(X) \neq P_T(X) \tag{5.4}$$

[Assumption: The conditional relationship $P(Y|X)$ is approximately shared across chemistry families. This is an idealization. The physics of chemical bonding applies everywhere, but different chemistry families may follow different structure-property relationships — e.g., the relationship between coordination number and band gap may differ between ionic halides and covalent nitrides. Such differences constitute concept shift. Phase A probed this assumption empirically: the asymmetric mixed-training recovery (§5.5 and §11 Finding 5 — 70–86% for formation energy vs. 46–58% for band gap) is consistent with mild concept shift on formation energy and stronger concept shift on band gap.]

Domain-shift split design. The source domain is oxides (the largest family in Materials Project). The OOD test sets are sulfides, nitrides, and halides. We hypothesized a difficulty gradient:

- Sulfides may be chemically closest to oxides (same group 16, similar bonding)
- Nitrides may be more different (group 15, different coordination preferences)
- Halides may be most different (group 17, highly ionic, different crystal structures)

Measured in Phase A (§11, Finding 3). The gradient was confirmed for formation energy: MAE degradation ratios of 2.3× (sulfide), 4.8× (nitride), 6.3× (halide), matching the hypothesized ordering exactly. For band gap, the gradient was weaker and partially scrambled (1.1× / 1.4× / 2.4×), consistent with composition features transferring better for electronic properties. The Phase B ALIGNN model partially disrupted this pattern with a sulfide-specific systematic bias (§11 Finding 10) that is not predicted by the chemical-distance argument — a surprising negative result.

5.3 Standard Split

Standard split: 70/10/10/10 (train/val/cal/test) stratified by chemistry family and discretized target. The stratification ensures each split preserves the family proportions and target distribution of the full dataset.

CI regression fixture. Separately, a small **100-crystal frozen fixture** (25 per chemistry family, cached as parquet + pickle) is used for CI regression gating. This is not one of the standard splits — it is a fixed, small test set designed to catch regressions quickly. The MAE on this fixture must remain below a stored threshold across code changes.

Discretization for stratification. The continuous target is binned into 10 equal-frequency bins before stratification. This prevents pathological splits where rare property values concentrate in one partition.

5.3.1 Target-Quartile Stratification for the Domain-Shift Split (DECISIONS.md topic 16)

The domain-shift split trains exclusively on oxides, so **chemistry-family stratification is degenerate within the oxide-only training pool**. An initial implementation omitted stratification inside `domain_shift_split`, reasoning that the family axis was constant. This caused a measurable problem: Tier 2

domain-shift formation energy showed $R^2 = 0.837 \pm 0.103$ across 3 seeds, with per-seed ID MAE ranging from 0.117 to 0.150 — roughly a 28% spread that is far larger than the corresponding Tier 1 spread.

Root cause. With ~15K oxides distributed across a 70/10/10/10 split, different random seeds placed "easy" oxides (typical well-coordinated structures) and "hard" oxides (distorted, low-symmetry, or chemically unusual compositions) into very different partitions. R^2 is especially sensitive because its denominator is the total target variance within the test set — a single seed that concentrated hard oxides in the test set showed both higher MAE and lower R^2.

Fix: target-quartile stratification. We bin the continuous target value (formation energy or band gap, whichever is being predicted) into quartiles via `pd.qcut(y, q=4)` and pass the quartile labels as the `stratify=` argument to all three `train_test_split` calls in `domain_shift_split`. This ensures each partition contains roughly equal proportions of the four target-quartile bins, spreading easy and hard oxides evenly across train/val/cal/test.

Theoretical justification. Target-quartile stratification is a variance-reduction technique for the sampling distribution of finite-sample metrics. By enforcing balanced target-distribution statistics across partitions, it reduces the seed-to-seed variance of R^2 without changing the expected value of any metric. It is the within-family analog of the chemistry-family stratification used in the standard split (§5.3). The fallback behavior — if any quartile bin has fewer than 2 members, the function reverts to unstratified splitting — handles small fixtures and edge cases gracefully.

Measured effect. After applying target-quartile stratification, Tier 2 domain-shift formation energy variance dropped from R^2 std 0.103 to 0.048 (53% reduction). Combined with increasing the oxide subsample from 15K to 25K (DECISIONS.md topic 15), the final variance is R^2 std 0.040 and MAE std 0.002 — matching Tier 1's stability profile. See DECISIONS.md topic 16 for the full stage-by-stage measurements.

Why not simply add more seeds? Running 5 or 10 seeds instead of 3 would reduce the reported std mechanically but would not fix the underlying problem — each seed still samples from a broken partition-quality distribution. Stratification fixes the partition quality; averaging over more seeds would only reduce the noise in our estimate of the broken distribution's std.

Implemented in `src/crystal_prop_bench/data/splits.py::domain_shift_split()`. The `stratify_col` parameter controls which column is used for quartile binning (defaulting to the target column). Tests in `tests/test_splits.py` verify that stratification preserves target-distribution quartile proportions across partitions within a tolerance of ±5%.

5.4 Degradation Metrics

For source domain (oxides) with test MAE $\mathrm{MAE}_{\mathrm{ID}}$ and target domain k with test MAE $\mathrm{MAE}_{\mathrm{OOD}}^{(k)}$, define the **degradation ratio**:

$$\Delta_k = \frac{\mathrm{MAE}_{\mathrm{OOD}}^{(k)}}{\mathrm{MAE}_{\mathrm{ID}}} \tag{5.5}$$

$\Delta_k > 0$ by construction. $\Delta_k = 1$ means no degradation; $\Delta_k > 1$ means the model performs worse on family k; $\Delta_k < 1$ would mean the model performs *better* on the OOD family than on ID, which is possible if the OOD family is intrinsically easier to predict. This ratio is comparable across targets with different scales.

5.5 Domain Randomization as Mitigation

The **mixed-train** split trains on all chemistry families combined (70/10/10/10 stratified), then evaluates per-family performance. This was designed to test whether **domain randomization** — exposing the model to diverse training domains — is an effective mitigation for distribution shift, paralleling the B3 finding in sim-to-data.

Formally, domain randomization replaces $P_S(X)$ with a mixture:

$$P_{\text{mixed}}(X) = \sum_k w_k \, P_k(X) \tag{5.6}$$

where w_k is the proportion of family k in the training set. If the conditional relationship $P(Y|X)$ is approximately shared across families (mild concept shift), then training on the mixture should generalize better than training on any single family.

Evaluation criterion. We compare: - $\text{MAE}_{\text{OOD}}^{(k)}$ from the oxide-only model - $\text{MAE}_{\text{mixed}}^{(k)}$ from the mixed-train model evaluated on the same OOD family k

Measured result (asymmetric recovery). Mixed training recovered 70–86% of formation-energy degradation but only 46–58% of band-gap degradation. Halides showed the strongest formation-energy recovery (86%). See §11 Finding 5 for the full numbers.

Interpretation. The asymmetry is predicted by the concept-shift argument in §5.2. Formation energy depends on chemical-bonding energetics — a local, roughly transferable phenomenon across chemistry families — so $P(Y|X)$ is approximately shared and the mixture $P_{\text{mixed}}(X)$ captures most of the relevant training distribution. Band gap depends on electronic structure, which differs more fundamentally across families (different orbital hybridizations, different gap mechanisms in halides vs. nitrides vs. oxides). For band gap, the concept-shift component is larger, and pooling families yields smaller gains because the learned function $\hat{f}$ must compromise across incompatible regimes. This is consistent interpretation rather than falsification — domain randomization helps when concept shift is mild and helps less when concept shift is strong.

Domain-shift splits and the standard split are implemented in `src/crystal_prop_bench/data/splits.py`*. The chemistry-family classifier is a standalone function in* `src/crystal_prop_bench/data/chemistry.py::clas` `chemistry_family()`*, importable from evaluation and plotting modules. Degradation metrics and domain-shift analysis are in* `src/crystal_prop_bench/evaluation/domain_shift.py`*.*

5.6 Featurization Bias Check

The Voronoi subsampling of §2.2.4 raises a concern: if the ~58K Voronoi-survivable subset is systematically different from the full ~110K corpus (e.g., if Voronoi fails preferentially on certain chemistry families or structural types), then the Tier 2 benchmark is not directly comparable to Tier 1. A model trained on an easier subset would report artificially good MAE.

To detect this, we evaluate Tier 1 on **both** datasets:

- `tier1_full`: Tier 1 (Magpie) trained and tested on the full classified corpus (~110K crystals)
- `tier1_voronoi_subset`: Tier 1 (Magpie) trained and tested on the Voronoi-survivable subset (~58K crystals)

Both results appear as separate rows in `results/tables/benchmark.csv`. The comparison answers:

- **If** $\text{MAE}_{\text{tier1_voronoi_subset}} \approx \text{MAE}_{\text{tier1_full}}$: Voronoi failures do not select a systematically easier or harder subset. The drop-and-report strategy is validated, and Tier 1 vs. Tier 2 comparisons on the subset are interpretable.
- **If** the two Tier 1 MAEs differ substantially: the Voronoi failure set is informative and encodes a hidden selection effect. Any Tier 2 improvement over Tier 1 must be decomposed into (a) improvement from structural features and (b) improvement from evaluating on an easier subset. The bias check makes the potential confound visible rather than hiding it.

This bias check is a correctness tool that was added mid-Phase-A when the oxide subsampling decision was made. It costs one extra Tier 1 training run per target × seed and gives a clean decomposition of selection vs. architectural effects.

Implemented in `scripts/run_tier1.py` (which trains on both `tier1_full` and `tier1_voronoi_subset` configurations) and `src/crystal_prop_bench/evaluation/metrics.py` (which reports both in the benchmark table).

---

6. Explainability: Shapley Values and TreeSHAP

6.1 Shapley Values: From Game Theory to Feature Attribution

Shapley values (Shapley, 1953) originate in cooperative game theory. Consider a game with d players (features) and a value function $v : 2^{\{1,\ldots,d\}} \to \mathbb{R}$ mapping subsets of players to payoffs. The **Shapley value** of player j is:

$$\phi_j(v) = \sum_{S \subseteq \{1,\ldots,d\} \setminus \{j\}} \frac{|S|!\,(d - |S| - 1)!}{d!} \left[v(S \cup \{j\}) - v(S) \right] \tag{6.1}$$

This is the average marginal contribution of player j across all possible orderings in which players can join the coalition.

Shapley's theorem *[Exact]* states that ϕ_j is the *unique* attribution satisfying four axioms:

1. **Efficiency:** $\sum_{j=1}^{d} \phi_j = v(\{1, \ldots, d\}) - v(\emptyset)$ — attributions sum to the total effect
2. **Symmetry:** If j and k contribute equally to all coalitions, $\phi_j = \phi_k$
3. **Null player:** If j contributes nothing to any coalition, $\phi_j = 0$
4. **Linearity:** Shapley values of a sum of games equal the sum of Shapley values

6.2 SHAP: Shapley Values for ML

SHAP (SHapley Additive exPlanations; Lundberg & Lee, 2017) adapts Shapley values to ML by defining the value function. The general SHAP framework defines $v(S)$ as the expected model output conditioned on the features in S:

$$v(S) = \mathbb{E}\left[\hat{f}(X) \mid X_S = \mathbf{x}_S \right] \tag{6.2}$$

where X_S denotes the subset of features indexed by S, fixed to their observed values $\mathbf{x}_S$. The resulting SHAP value $\phi_j(\mathbf{x})$ decomposes the model's prediction for input $\mathbf{x}$ into per-feature contributions:

$$\hat{f}(\mathbf{x}) = \phi_0 + \sum_{j=1}^{d} \phi_j(\mathbf{x}) \tag{6.3}$$

where $\phi_0 = \mathbb{E}[\hat{f}(X)]$ is the base value (mean prediction). This decomposition satisfies the efficiency axiom by construction.

[Important caveat: Equation (6.2) defines the value function using conditional expectations, which is one of several possible SHAP formulations. In practice, the exact semantics depend on how absent features are handled. For tree-based models, we use TreeSHAP, which implements a specific tree-path-dependent formulation — see §6.3.]

6.3 TreeSHAP: Exact Computation for Tree Ensembles

Computing Equation (6.1) naively requires $O(2^d)$ evaluations of v, which is intractable for $d \approx 150$–250 features. **TreeSHAP** (Lundberg et al., 2020) exploits the tree structure of gradient boosting models to compute Shapley values in $O(TLD^2)$ time, where T is the number of trees, L is the maximum number of leaves per tree, and D is the maximum depth.

The key insight is that decision trees partition the feature space into axis-aligned rectangles (leaves), and the value function $v(S)$ can be computed by tracking which branches are taken (for features in S) versus marginalized over (for features not in S).

Tree-path-dependent semantics. *[Assumption]* TreeSHAP, as implemented in the `shap` library for tree models, uses a **tree-path-dependent** formulation of the value function. This is neither the pure conditional expectation of Equation (6.2) nor a purely interventional (marginal) formulation. Instead, it traces the actual tree paths: for features in S, the tree follows the observed split; for features not in S, the tree distributes probability across child nodes proportionally to the training data that reached each child. This is exact for the chosen formulation — there is no sampling or approximation involved — but the resulting attributions have subtly different semantics from the conditional-expectation SHAP values defined in (6.2). For correlated features (common in Magpie descriptors, where many elemental properties are correlated), tree-path-dependent SHAP values may attribute importance differently than conditional or interventional formulations.

We use tree-path-dependent TreeSHAP because: (a) it is the default in the `shap` library for tree models, (b) it is computationally exact and fast, and (c) the per-family comparison (§6.4) uses the same formulation throughout, so relative comparisons are internally consistent even if the absolute attribution semantics differ from the conditional-expectation ideal.

6.4 Per-Family Feature Importance Comparison

For each chemistry family $\mathcal{F}_k$, compute the global feature importance as the mean absolute SHAP value:

$$I_j^{(k)} = \frac{1}{|\mathcal{F}_k|} \sum_{\mathbf{x} \in \mathcal{F}_k} |\phi_j(\mathbf{x})| \tag{6.4}$$

Comparing $I_j^{(k)}$ across families reveals which features are universally important vs. family-specific. If the model relies on different features for different families, this explains why domain-shift degrades performance — the model's learned feature-property mapping is family-dependent.

6.5 Failure-Case Characterization

For each tier, extract the 50 worst-predicted crystals (largest $|y - \hat{f}(\mathbf{x})|$). Characterize failures by:

- Chemistry family distribution: are failures concentrated in specific families?
- Space group distribution: do certain structural types cause systematic failures?
- Number of atoms per unit cell: do complex structures with many atoms fail more?
- Cross-tier overlap: if both Tier 1 and Tier 2 fail on the same crystals, the failure is composition-based; if Tier 2 rescues Tier 1 failures, structure information was the missing ingredient.

SHAP analysis is implemented in `src/crystal_prop_bench/evaluation/explainability.py`. *Training script:* `scripts/run_shap.py`. *Visualization:* `src/crystal_prop_bench/visualization/plots.py`.

---

7. Crystal Graph Neural Networks (Phase B: ALIGNN)

This section derives the ALIGNN representation and architecture. Phase B was originally planned around vanilla CGCNN (Xie & Grossman, 2018) but was revised mid-project to use **ALIGNN** (Atomistic Line Graph Neural Network; Choudhary & DeCost, 2021), which extends CGCNN's atom-graph convolution with a second graph — the **line graph** — whose message passing encodes bond-angle triplets. The rationale is documented in DECISIONS.md (topic 17): for the CuspAI application specifically, angular features provide a more convincing geometric-deep-learning story than distance-only edge features, and connect naturally to the equivariant architecture family (MACE, NequIP) used in production crystal generation pipelines.

7.1 Why Angular Features

Physical assumption. *[Assumption]* Beyond pairwise distances, the geometry of a crystal is characterized by **bond-angle triplets** — the angles formed by three atoms connected through bonds (i, j) and (j, k). These angles distinguish crystal structures that share composition and coordination number but have different local geometries (e.g., cubic vs. hexagonal close-packed, octahedral vs. trigonal prismatic coordination). A distance-only graph convolution (CGCNN) cannot distinguish these directly — two crystals with identical pair-distance distributions but different angular arrangements would receive the same edge features.

ALIGNN addresses this by constructing a second graph — the **line graph** — whose nodes are the atom-graph edges (bonds) and whose edges are the bond-angle triplets. Running message passing on both graphs in alternation injects angular information into atom representations without breaking rotational invariance.

7.2 Atom Graph and Line Graph Construction

Atom graph. The atom graph $\mathcal{G}_a = (\mathcal{V}_a, \mathcal{E}_a)$ is constructed exactly as in CGCNN:

- **Nodes** $\mathcal{V}_a = \{1, \dots, N_{\text{atoms}}\}$: atoms in the unit cell, with initial features encoding element identity

- **Edges** $\mathcal{E}_a$: directed pairs (i, j) where atom j is among the $K_{\text{nn}} = 12$ nearest neighbors of atom i within cutoff $r_c = 8.0$ Å (including periodic images)

The 12-nearest-neighbor cap (DECISIONS.md topic 18) controls graph density across structures with varying coordination environments — without it, large unit cells produce graphs with thousands of edges and training becomes memory-bound.

Each edge carries a scalar distance d_{ij}, expanded via Gaussian radial basis functions:

$$e_{ij,k} = \exp\left(-\frac{(d_{ij} - \mu_k)^2}{2\sigma^2}\right), \quad k = 0, 1, \ldots, K_e - 1 \tag{7.1}$$

with $K_e = 80$ centers uniformly spaced from 0 to r_c. The output $\mathbf{e}_{ij} \in \mathbb{R}^{80}$ is the initial edge feature.

Line graph. The line graph $\mathcal{L}(\mathcal{G}_a) = (\mathcal{V}_\ell, \mathcal{E}_\ell)$ is a new graph constructed from $\mathcal{G}_a$:

- **Nodes** $\mathcal{V}_\ell = \mathcal{E}_a$: each atom-graph *edge* becomes a line-graph *node*. Initial features of line-graph node (i, j) are the atom-graph edge features $\mathbf{e}_{ij}$.
- **Edges** $\mathcal{E}_\ell$: two line-graph nodes (i, j) and (j, k) are connected in $\mathcal{L}(\mathcal{G}_a)$ iff they share a common atom j in the atom graph. Each line-graph edge corresponds to a **bond-angle triplet** (i, j, k) — the angle θ_{ijk} subtended at atom j between bonds (i, j) and (j, k).

Triplet angle encoding. Each line-graph edge carries a scalar angle

$$\theta_{ijk} = \arccos\left(\frac{\mathbf{r}_{ji} \cdot \mathbf{r}_{jk}}{\|\mathbf{r}_{ji}\| \, \|\mathbf{r}_{jk}\|}\right) \tag{7.2}$$

where $\mathbf{r}_{ji} = \mathbf{r}_i - \mathbf{r}_j$ and $\mathbf{r}_{jk} = \mathbf{r}_k - \mathbf{r}_j$. The angle is expanded via a cosine/radial basis function into a $K_\theta = 40$-dimensional feature vector. The specific expansion is inherited from the `alignn` package implementation and uses RBFs over $\cos\theta_{ijk} \in [-1, 1]$.

The resulting crystal representation is a pair $(\mathcal{G}_a, \mathcal{L}(\mathcal{G}_a))$, constructed by `graph_builder.py` and cached to disk before training.

7.3 Invariance Properties

Rotational invariance. *[Exact]* Interatomic distances d_{ij} and bond angles θ_{ijk} are **rotationally invariant scalar quantities** — they do not change under global rotations of the crystal. Since ALIGNN operates only on these scalar features (never on raw coordinates or vectors), every operation in the forward pass is rotationally invariant by construction. The predicted property $\hat{y}$ is therefore invariant under $\mathrm{SO}(3)$ rotations of the input structure.

Translational invariance. Similarly, distances and angles depend only on relative positions, so translational invariance is automatic.

Permutation invariance. The final global readout (mean pooling, §7.5) is a symmetric function of atom features, so the prediction does not depend on the order in which atoms are listed. Combined with rotational and translational invariance, ALIGNN respects all the symmetries of the scalar target properties E_f and E_g.

[Note on equivariant architectures: ALIGNN is invariant, not equivariant. Fully equivariant architectures (MACE, NequIP, Equiformer) preserve the transformation behavior of vector and tensor quantities under rotations, which matters when the target itself is a vector or tensor (e.g., forces, dipole moments). For scalar targets, invariance is sufficient and ALIGNN's scalar-feature design is a reasonable choice. The intellectual bridge to equivariant architectures is that both families respect physical symmetries; ALIGNN achieves this through scalarization, equivariant models through explicit group-theoretic construction.]

7.4 Alternating Message Passing

ALIGNN interleaves convolutions on the two graphs. One ALIGNN layer consists of a **line-graph update** followed by an **atom-graph update**.

Line-graph update (edge-gated convolution over triplets). Each line-graph node (i, j) (i.e., each atom-graph edge) receives messages from its neighbors in $\mathcal{L}(\mathcal{G}_a)$ — which are the other bonds sharing an atom with (i, j). The update uses the gated convolution form (generalization of Xie & Grossman 2018's equation, with triplet features added):

$$\mathbf{e}_{ij}^{(l+1)} = \mathbf{e}_{ij}^{(l)} + \sum_{(j,k)\in\mathcal{N}_\ell(i,j)} \sigma\Big(\mathbf{W}_f^\ell\,[\mathbf{e}_{ij}^{(l)} \oplus \mathbf{e}_{jk}^{(l)} \oplus \mathbf{t}_{ijk}]\Big) \odot g\Big(\mathbf{W}_s^\ell\,[\mathbf{e}_{ij}^{(l)} \oplus \mathbf{e}_{jk}^{(l)} \oplus \mathbf{t}_{ijk}]\Big) \tag{7.3}$$

where:

- $\mathcal{N}_\ell(i, j)$ is the line-graph neighborhood of node (i, j)
- $\mathbf{t}_{ijk} \in \mathbb{R}^{40}$ is the triplet angular feature (RBF expansion of $\cos\theta_{ijk}$)
- σ is sigmoid (gate), g is SiLU/softplus
- $\mathbf{W}_f^\ell, \mathbf{W}_s^\ell$ are learnable matrices for the filter and self branches
- $\oplus$ denotes concatenation, $\odot$ elementwise product

This update injects angular information into the bond representations: a bond between two atoms "learns" not just its own distance but also what angles it forms with neighboring bonds.

Atom-graph update (standard CGCNN-style gated convolution). After the line-graph update produces refreshed edge features $\mathbf{e}_{ij}^{(l+1)}$, each atom node is updated using those edge features:

$$\mathbf{v}_i^{(l+1)} = \mathbf{v}_i^{(l)} + \sum_{j\in\mathcal{N}_a(i)} \sigma\Big(\mathbf{W}_f^a\,[\mathbf{v}_i^{(l)} \oplus \mathbf{v}_j^{(l)} \oplus \mathbf{e}_{ij}^{(l+1)}]\Big) \odot g\Big(\mathbf{W}_s^a\,[\mathbf{v}_i^{(l)} \oplus \mathbf{v}_j^{(l)} \oplus \mathbf{e}_{ij}^{(l+1)}]\Big) \tag{7.4}$$

This is structurally identical to CGCNN's gated convolution (the equation we derived in the original §7 for vanilla CGCNN), **except that the edge features $\mathbf{e}_{ij}^{(l+1)}$ now carry angular information** from the preceding line-graph update. The alternating pattern is what distinguishes ALIGNN from CGCNN: angular information flows from triplets → bonds → atoms.

Layer count (DECISIONS.md topic 22). The published ALIGNN configuration uses 4 ALIGNN layers (each with a line-graph update followed by an atom-graph update) followed by 4 additional atom-graph-only GCN layers. For training-cost reasons documented in DECISIONS.md topic 22, this project uses a **3+3 configuration** (3 ALIGNN layers + 3 GCN layers), which captures most of the angular-feature benefit at approximately 60% compute. The depth reduction was chosen before training and held fixed across all seeds and splits.

7.5 Graph-Level Readout

For property prediction, the final atom features are pooled into a single crystal-level vector via **global mean pooling**:

$$\mathbf{u} = \frac{1}{N_{\text{atoms}}} \sum_{i=1}^{N_{\text{atoms}}} \mathbf{v}_i^{(L)} \tag{7.5}$$

Mean pooling (rather than sum) is appropriate because formation energy per atom and band gap are both **intensive properties** — they do not scale with system size. Sum pooling would create a spurious correlation with unit-cell size, biasing predictions for larger cells. This choice is documented in DECISIONS.md (topic 19).

The crystal-level vector is passed through an MLP head producing the scalar prediction $\hat{y}$.

7.6 Training Objective

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{(\mathcal{G}_i, y_i) \in \mathcal{B}} (y_i - \hat{y}_i)^2 + \lambda \|\theta\|_2^2 \tag{7.6}$$

where $\mathcal{B}$ is a mini-batch, θ collects all learnable parameters, and λ is the weight-decay coefficient. Optimized with AdamW and OneCycleLR (the ALIGNN package defaults, with epoch count and patience tuned for compute as described in DECISIONS.md topic 22).

7.7 Conformal Prediction on GNN Outputs

Split conformal prediction (§4) applies identically to ALIGNN outputs — it is model-agnostic. The calibration residuals $R_i = |Y_i - \hat{f}_{\text{ALIGNN}}(X_i)|$ have a different distribution than the LightGBM residuals (in particular, they are smaller on the standard split — see §11 Finding 8), which *in principle* yields tighter conformal intervals at matched coverage. We do not report Tier 3 interval widths quantitatively in the benchmark tables for Phase A, so the practical size of this effect is left as an observation rather than a measured comparison.

More importantly, the per-family recalibration theorem in §4.7 applies identically to any frozen model. The architecture-agnostic recipe ($n_{\text{cal}} \approx 25$ OOD samples restore coverage) works for ALIGNN just as it does for Tiers 1 and 2. This is the content of §11 Finding 11 and is the strongest reason the calibration sweep is the repo's headline deployable result: it transfers across architectures without modification.

7.7.1 ALIGNN Does Not Solve Chemistry Shift — The Sulfide Bias

One of the central negative results of Phase B is that **ALIGNN does not solve the chemistry-shift problem**, despite its substantially better ID accuracy. Under the oxide→OOD shift protocol, Tier 3 formation-energy degradation is 15.7× on sulfides, 6.5× on nitrides, and 4.5× on halides. On nitrides and halides, ALIGNN actually beats Tier 1 OOD — so the failure is **sulfide-specific, not architectural**.

Diagnostic analysis of the sulfide failure (§11 Finding 10) shows the prediction distribution is **shifted, not collapsed**. On sulfide OOD test points: `y_true` mean = -1.120 eV/atom, `y_pred` mean = -0.223 eV/atom — a systematic bias of $+0.897$ eV/atom. The predicted variance is 76% of the true variance,

so the outputs retain reasonable spread; they are simply offset. **Interpretation:** ALIGNN learned oxide-specific energy scales from the training distribution and cannot shift them for sulfides, despite having no explicit architectural bias toward oxides. This is a form of implicit covariate-shift failure in the learned representation.

Two points about what this does and does not say:

1. **It does not falsify ALIGNN's geometric benefit on the standard split.** The 51% ID formation-energy improvement is real (§11 Finding 8). The scientifically interesting result is that a strong ID improvement does not automatically transfer OOD — the inductive bias that helps on one distribution can hurt on another.
2. **The calibration sweep (§4.7) handles the failure at deployment time.** A practitioner encountering sulfides can recalibrate with ~25 DFT-validated sulfide samples and regain coverage (Finding 11), even though the *point predictions* remain biased. This is the deployable recipe the repo delivers. The bias itself is a fundamental limitation of architecture-level transfer and not a conformal-prediction failure.

The 3+3 layer configuration (DECISIONS.md topic 22) was chosen for training cost, not scientific reasons. Whether the published 4+4 depth would exhibit the same sulfide bias is an open question — this benchmark does not resolve it.

7.8 Implementation Approach

Hybrid implementation (DECISIONS.md topic 20). We import the ALIGNN architecture class from the `alignn` package (NIST/JARVIS implementation, DGL backend) but own the rest of the pipeline:

- **Graph construction** (`src/crystal_prop_bench/data/graph_builder.py`): converts pymatgen `Structure` objects into `(atom_graph, line_graph)` DGL tuples, caches to disk
- **Training loop** (`scripts/run_tier3.py`): our own loop, MLflow logging, prediction export
- **Prediction export**: writes to the standard `results/predictions/tier3_*.parquet` format, identical schema to Tiers 1–2

This keeps the evaluation seam clean: `run_evaluation.py` reads Tier 3 parquets alongside Tier 1 and Tier 2 parquets without modification. The benchmark table, conformal figures, and domain-shift figures grow automatically to include Tier 3 rows.

Element vocabulary. The `alignn` package provides its own atomic feature table (derived from CGCNN). We inherit this vocabulary rather than maintaining our own.

Training hardware. A100 80GB via Modal (DECISIONS.md topic 21). Tested on smaller GPUs and found OOM at batch_size=128 on T4, A10G, and A100 40GB — the line-graph doubles memory usage relative to CGCNN, and the few largest structures in each batch dominate peak memory.

ALIGNN is implemented in `src/crystal_prop_bench/models/alignn_wrapper.py` (importing the `alignn` package's architecture). Graph construction: `src/crystal_prop_bench/data/graph_builder.py`. Training: `scripts/run_tier3.py` with Modal entrypoint for A100 80GB. Graph construction tests: `tests/test_graph_builder.py`.

---

8. Reference Paper Connections

8.1 Ward et al. (2016) — "A general-purpose machine learning framework for predicting properties of inorganic materials"

- **We use:** Magpie composition descriptors as Tier 1 features
- **Their result:** Composition-only random forest achieves MAE ~0.1 eV/atom on formation energy for ICSD entries
- **Our extension:** We evaluate on Materials Project (~150K compounds vs. their ~20K), add structural descriptors (Tier 2), and layer conformal prediction + domain-shift analysis on top
- **Validation target:** Our Tier 1 MAE should be competitive with their reported values, adjusted for dataset differences

8.2 Choudhary & DeCost (2021) — "Atomistic Line Graph Neural Network for improved materials property predictions"

- **Primary Phase B reference.** *npj Computational Materials* 7, 185.
- **We use:** ALIGNN architecture for Tier 3 — the line graph construction (§7.2), alternating line-graph/atom-graph message passing (§7.4, Equations 7.3–7.4), and the hybrid scalar-feature invariant design (§7.3)
- **Their result:** MAE 0.022 eV/atom on Materials Project formation energy, 0.218 eV on band gap (published 4+4 layer configuration, full training schedule)
- **Our numbers:** MAE 0.051 eV/atom formation energy, 0.337 eV band gap (3+3 layer configuration, 80 epochs, single seed — see DECISIONS.md topic 22 for the training-cost rationale)
- **Differences from their paper:** We use their architecture but (a) apply a reduced-depth configuration for training-cost reasons, (b) add conformal prediction with three evaluation modes (§4.8), (c) add domain-shift evaluation across chemistry families, and (d) run the per-family calibration sweep (§4.7) to produce the deployable UQ recipe
- **Implementation note:** We import the ALIGNN model class from the NIST/JARVIS `alignn` package (DGL backend) and own the rest of the pipeline (graph construction, training loop, prediction export) — see DECISIONS.md topic 20

8.2b Xie & Grossman (2018) — "Crystal Graph Convolutional Neural Networks for an Accurate and Interpretable Prediction of Material Properties" (historical reference)

- **Historical relevance.** *Physical Review Letters* 120, 145301. The progenitor of gated graph convolution for crystal property prediction; ALIGNN's atom-graph update (§7.4, Equation 7.4) directly inherits the gated-convolution form from this paper
- **Their Eq. (3):** Original CGCNN gated convolution — our Equation (7.4) is the same form, with the critical difference that our edge features $\mathbf{e}_{ij}$ carry angular information from the line-graph update in ALIGNN
- **Their result:** MAE 0.039 eV/atom on Materials Project formation energy, 0.388 eV on band gap
- **Why we did not use vanilla CGCNN.** Phase B was originally planned around CGCNN but revised to ALIGNN mid-project (DECISIONS.md topic 17). The CuspAI application specifically benefits from a geometric-deep-learning story built on angular features rather than distance-only convolutions; vanilla CGCNN would read as "learned the 2018 baseline" rather than "understands the progression toward equivariant architectures"

8.3 Vovk et al. (2005) — "Algorithmic Learning in a Random World"

- **We use:** Split conformal prediction framework — the theoretical foundation for §4
- **Their framework:** Conformal prediction with arbitrary nonconformity scores
- **Our specialization:** Absolute residuals as scores, applied to materials property regression
- **Key theorem:** Marginal coverage guarantee under exchangeability (our Theorem in §4.4)

8.4 Lei et al. (2018) — "Distribution-Free Predictive Inference for Regression"

- **We use:** Split conformal regression intervals — practical algorithm from their Section 2
- **Their contribution:** Formalized split conformal for regression with explicit finite-sample bounds
- **Their Eq. (2.3):** Defines the conformal quantile as the $\lceil (1-\alpha)(n+1) \rceil$-th smallest residual — our Equations (4.4)–(4.5) follow this convention exactly
- **Our extension:** We explicitly test the *breakdown* of the coverage guarantee under domain shift, which their theory predicts (since exchangeability is violated)
- **Implementation convention:** We implement their order-statistic definition using `np.quantile(..., method="higher")` — see §4.3

8.4b Tibshirani et al. (2019) — "Conformal Prediction Under Covariate Shift"

- **Primary reference for §4.7.** *NeurIPS 2019.*
- **We use:** Their shifted-exchangeability framework as the theoretical basis for per-family recalibration (§4.7)
- **Their contribution:** Generalized conformal prediction from exchangeable data to arbitrary test-time distribution shifts via importance-weighted calibration
- **Our specialization:** Our setting is a simpler special case — rather than reweighting a globally-calibrated interval, we *partition* the test distribution into chemistry families and apply standard split conformal within each family. This avoids importance-weight estimation at the cost of requiring a per-family calibration set. The §4.7 theorem is their result applied to the partition case.
- **Application:** Justifies the calibration sweep experiment (§4.8 mode 3) and Finding 6

8.5 Lundberg et al. (2020) — "From local explanations to global understanding with explainable AI for trees"

- **We use:** TreeSHAP for tree-path-dependent Shapley value computation on LightGBM models
- **Their contribution:** Polynomial-time exact SHAP for tree ensembles using tree-path-dependent value functions
- **Our extension:** Per-family comparison of feature importance — asking whether the same features matter for different chemistry domains
- **Convention note:** We use their tree-path-dependent formulation (the default in the `shap` library), not the conditional-expectation formulation. See §6.3 for the distinction.

8.6 Ke et al. (2017) — "LightGBM: A Highly Efficient Gradient Boosting Decision Tree"

- **We use:** LightGBM as the regression model for Tiers 1–2
- **Their contributions:** GOSS and EFB optimizations
- **Our configuration:** Phase A uses the default `boosting_type="gbdt"` (standard gradient boosting), not GOSS. EFB is active by default and may reduce the effective feature count for sparse Magpie features. See §3.2 for details.

- **Our parameters:** `num_leaves=127`, `learning_rate=0.05`, `objective="regression"` (squared-error loss), `metric="mae"` (early stopping on validation MAE)

8.7 Friedman (2001) — "Greedy function approximation: A gradient boosting machine"

- **We use:** Gradient boosting framework — Equations (3.4)–(3.6) follow his formulation
- **Their Eq. (10):** General gradient boosting update — our Equation (3.6)
- **Our application:** Materials property regression with composition and structure features

8.8 Friedman (2002) — "Stochastic gradient boosting"

- **We use:** Row subsampling (`subsample=0.8`) as a regularization technique
- **Their contribution:** Showed that training each tree on a random subsample of the data reduces variance and improves generalization
- **Our application:** Applied as `subsample=0.8` in the LightGBM configuration

8.9 Perdew, Burke & Ernzerhof (1996) — "Generalized Gradient Approximation Made Simple"

- **We use:** PBE-GGA as the DFT functional underlying the Materials Project dataset
- **Relevance:** PBE systematically underestimates band gaps by 30–50% relative to experiment. This systematic bias is consistent across the dataset and does not affect relative model comparisons, but is noted as a limitation in the README.
- **Convention note:** Our band gap targets are Kohn-Sham PBE gaps, not experimental gaps.

8.10 Shapley (1953) — "A value for n-person games"

- **We use:** Shapley value axioms (§6.1) as the theoretical foundation for SHAP
- **Their contribution:** Uniqueness theorem — the four axioms uniquely determine the attribution (our Equation 6.1)

8.11 Lundberg & Lee (2017) — "A Unified Approach to Interpreting Model Predictions"

- **We use:** SHAP framework connecting Shapley values to ML model explanations
- **Their contribution:** Defined the SHAP value function $v(S)$ as conditional expectation (our Equation 6.2) and showed the connection to additive feature attribution
- **Convention note:** The conditional-expectation definition in (6.2) is the general SHAP formulation. Our implementation uses TreeSHAP's tree-path-dependent variant (§6.3, §8.5), which differs subtly.

8.12 Barber et al. (2021) — "The limits of distribution-free conditional predictive inference"

- **We use:** Their impossibility result for distribution-free conditional coverage (cited in §4.5)
- **Their contribution:** Showed that no method can achieve conditional coverage $P(Y \in C(X) | X \in \mathcal{A}) \geq 1 - \alpha$ for all measurable sets $\mathcal{A}$ without distributional assumptions
- **Our application:** This result motivates our per-family conditional coverage analysis — we expect the marginal guarantee to break conditionally, and the impossibility theorem explains why no simple fix exists.

8.13 Vovk (2012) — "Conditional validity of inductive conformal predictors"

- **We use:** Theoretical results on conditional coverage limitations (cited alongside Barber et al. in §4.5)
- **Their contribution:** Earlier analysis of when and why conditional coverage fails for conformal predictors

8.14 Chemistry-family classifier

- **Source:** Internal design decision, not derived from a specific paper
- **Rationale:** Anion-framework classification is standard in solid-state chemistry (oxide, sulfide, nitride, and halide families correspond to distinct structural chemistry traditions), but the specific 80% purity threshold and pooled halide set are project-specific choices
- **Documentation:** Full rationale is specified for `DECISIONS.md` (topic 3)

---

9. Theory-to-Deliverable Mapping

This section maps every substantive theory result to the concrete repository deliverable that implements it. Phase A and Phase B are both shipped; file paths reflect the actual repository structure.

9.1 Code Deliverables

Theory Section	Key Equations	Shipped Deliverable	Application Claim
§1.1 (Crystal properties)	(1.1), (1.2)	`src/crystal_prop_bench/data/mp_adapter.py`	Correct DFT data ingestion, target definition
§1.1 (Data validation)	—	`src/crystal_prop_bench/data/schemas.py`	Pandera schema catches invalid entries, missing fields, out-of-range values
(Data access pattern)	—	`src/crystal_prop_bench/data/adapter.py`	`DatasetAdapter` ABC with 2 abstract methods (template-method pattern), portfolio-consistent
(Data download)	—	`scripts/download_data.py`	Reproducible data acquisition with caching
§5.1 (Chemistry classifier)	(5.1)–(5.3)	`src/crystal_prop_bench/data/chemistry.py::classify_chemistry_family()`	Standalone function (not an ABC method) — domain logic independent of data source
§2.1 (Magpie descriptors)	(2.1)–(2.3)	`src/crystal_prop_bench/data/featurizers.py`	Composition-only baseline with interpretable features; runs on ~110K full corpus
§2.2 (Voronoi coordination)	(2.4)–(2.5)	`src/crystal_prop_bench/data/featurizers.py`	Structure-aware features via parameter-free neighbor finding; parallelized via `featurize_dataframe(n_jobs=8)`
§2.2.4 (Oxide subsampling)	—	`src/crystal_prop_bench/data/featurizers.py`	Stratified oxide subsample (25K) + all minority families intact → ~58K Tier 2 dataset
§5.3 (Standard split)	—	`src/crystal_prop_bench/data/splits.py::standard_split()`	Stratified 70/10/10/10 by chemistry family and discretized target

Theory Section	Key Equations	Shipped Deliverable	Application Claim
§5.3.1 (Target-quartile stratification)	—	`src/crystal_prop_bench/data/splits.py::domain_shift_split()` (with `stratify_col`)	Reduces Tier 2 domshift R^2 variance from 0.103 → 0.048 (53% reduction)
§4.7 (Per-family recalibration protocol)	(4.9)–(4.11)	`src/crystal_prop_bench/data/splits.py::ood_calibration_sweep()`	Generates per-family cal/test partitions for $n_{\text{cal}} \in \{5, 10, 25, 50, 100\}$
§3 (Gradient boosting)	(3.4)–(3.6)	`src/crystal_prop_bench/models/lgbm_baseline.py`	Stateless `train_lgbm()` returning model + calibration residuals
§4.8 Mode 1 (Standard conformal)	(4.2)–(4.6)	`src/crystal_prop_bench/evaluation/conformal.py::conformal_interval()`	Distribution-free UQ with finite-sample coverage guarantee
§4.8 Mode 2 (Cross-domain conformal)	(4.8)	`src/crystal_prop_bench/evaluation/conformal.py::cross_domain_coverage()`	Demonstrates coverage collapse under exchangeability violation
§4.8 Mode 3 (Calibration sweep)	(4.9)–(4.11)	`src/crystal_prop_bench/evaluation/conformal.py::calibration_sweep()`	The deployable UQ recipe: $n_{\text{cal}} \approx 25$ restores coverage
§5.4 (Degradation metrics)	(5.5)	`src/crystal_prop_bench/evaluation/domain_shift.py`	Quantified OOD degradation per family
§5.5 (Domain randomization / mixed train)	(5.6)	`scripts/run_tier1.py` (mixed-split variant)	Empirical test of concept-shift asymmetry
§5.6 (Featurization bias check)	—	`scripts/run_tier1.py` (both `tier1_full` and `tier1_voronoi_subset` runs)	Detects selection effects from Voronoi failures
§3 (Aggregation)	—	`src/crystal_prop_bench/evaluation/metrics.py::aggregate_seeds()`	Mean ± std across 3 seeds, called from script layer
§3.1, Eq. (3.3) (Metrics)	(3.3)	`src/crystal_prop_bench/evaluation/metrics.py`	MAE, RMSE, R^2 overall and per-family breakdown
§6.2–6.3 (TreeSHAP)	(6.2)–(6.4)	`src/crystal_prop_bench/evaluation/explainability.py`	Per-feature attribution under tree-path semantics
§6.5 (Failure cases)	—	`src/crystal_prop_bench/evaluation/explainability.py`	50 worst crystals per tier, cross-tier overlap (Finding 7)
§7.1–7.2 (ALIGNN graphs)	(7.1)–(7.2)	`src/crystal_prop_bench/data/graph_builder.py`	Atom + line graph construction from pymatgen `Structure`, DGL format, cached
§7.4 (ALIGNN message passing)	(7.3), (7.4)	`src/crystal_prop_bench/models/alignn_wrapper.py`	Imports `alignn` package architecture (3+3 layer config, DECISIONS topic 22)
§7.5 (Mean pooling for intensive properties)	(7.5)	`src/crystal_prop_bench/models/alignn_wrapper.py`	Correct intensive-property readout
§7.6 (ALIGNN training)	(7.6)	`scripts/run_tier3.py`	Own training loop, MLflow logging, Modal A100 80GB entrypoint
§7.7 (GNN conformal)	—	`src/crystal_prop_bench/evaluation/conformal.py` (reused)	Architecture-agnostic UQ via prediction parquets

Theory Section	Key Equations	Shipped Deliverable	Application Claim
(Prediction interchange format)	—	`results/predictions/{tier}_{split}_seed{seed}_{target}_{subset}.parquet`	Clean seam between training and evaluation

9.2 Benchmark Artifacts

Theory Section	Shipped Artifact	Application Claim
§3, §4, §5	`results/tables/benchmark.csv` — Tier × Split × Target × MAE ± std (3 seeds Tier 1–2, single seed Tier 3)	Core deliverable; includes both `tier1_full` and `tier1_voronoi_subset` for bias check
§5.4	`results/tables/domain_shift.csv` + `results/figures/domain_shift_bars.png`	Per-family degradation ratios (Findings 3, 10)
§4.8 Modes 1–2	`results/tables/conformal_coverage.csv` + `results/figures/conformal_coverage.png`	Coverage × width at each α; shows ID vs. OOD collapse (Finding 4)
§4.7, §4.8 Mode 3	`results/tables/calibration_sweep.csv` + `results/figures/calibration_sweep.png`	**The standout figure** — coverage recovery curve per family (Finding 6)
§5.6	`results/tables/bias_check.csv`	`tier1_full` vs. `tier1_voronoi_subset` MAE comparison
§6.4	`results/figures/shap_summary.png`	Global feature importance, per-family comparison (supports Finding 7 interpretation)
§1.2	`README.md` §"Connection to Conditional Crystal Generation"	Articulates predictor role in CuspAI-style pipelines

9.3 Documentation Deliverables

Theory Section	Shipped Deliverable	Purpose
§1–§7 (all decisions)	`DECISIONS.md` (22 entries, topics 1–22)	Design rationale for every non-obvious choice, including mid-project revisions (Voronoi parallelization, oxide subsampling, stratification fix, CGCNN→ALIGNN pivot, depth 3+3 for cost control)
§3, §7	`MODEL_CARD.md`	Model documentation following Mitchell et al. (2019)
All	`README.md`	Motivation, 11 reported findings, figures, limitations, quick start
All	`Dockerfile`	Reproducibility
§3, §7	MLflow logging in training scripts	Experiment tracking during iteration; flat files (`results/tables/`, `results/figures/`) are the public interface (DECISIONS topic 12)
All	`.github/workflows/ci.yml`	ruff + mypy + tests + MAE regression gate on 100-crystal fixture

9.4 Test Deliverables

Theory Section	Shipped Test	What It Validates
§1.1, §2	`tests/test_schemas.py`	Pandera catches invalid entries, out-of-range values
§1.1	`tests/test_adapters.py`	MPAdapter with mocked API responses (`@pytest.mark.network` for live calls)
§5.1	`tests/test_chemistry.py`	Pure oxide → oxide; 79% → None; mixed halides → halide if combined ≥80%; OxState failure → dropped
§5.3, §5.3.1	`tests/test_splits.py`	Same seed → same split; target-quartile stratification preserves proportions; `ood_calibration_sweep` returns correct cal sizes
§2.1–2.2	`tests/test_featurizers.py`	Magpie output shape; Voronoi graceful failure; **exact Voronoi-surviving count pinned in fixture** (catches matminer regressions)
§4.3	`tests/test_conformal.py`	Synthetic residuals → correct empirical coverage; order-statistic convention exact on known vector; edge case $k > n$ returns infinite interval
§3	`tests/test_metrics.py`	`aggregate_seeds` exact mean/std; per-family breakdown
§6	`tests/test_integration.py`	SHAP runs on 100-crystal fixture; efficiency axiom $\sum_j \phi_j + \phi_0 = \hat{f}(\mathbf{x})$ holds within 10^{-6}
§7.1	`tests/test_graph_builder.py`	Atom graph + line graph construction: correct neighbor count, edge directionality, periodic images, line-graph node-count equals atom-graph edge count
All	`tests/test_integration.py`	100-crystal fixture → featurize → train → predict → evaluate end-to-end

---

10. Validation Criteria

This section defines **correctness gates** — conditions that must hold for the implementation to be considered correct. Directional hypotheses about empirical findings are in §11.

10.1 Data Pipeline Correctness

Criterion	Requirement	Validation
Formation energy target	Matches Materials Project `formation_energy_per_atom` field exactly	`tests/test_adapters.py`
Band gap target	Non-negative for all entries	`tests/test_schemas.py`
Chemistry classifier: pure oxide	Classified as oxide	`tests/test_chemistry.py`
Chemistry classifier: mixed < 80%	Dropped	`tests/test_chemistry.py`
Chemistry classifier: halide pooling	50% F + 40% Cl → halide (90% ≥ 80%)	`tests/test_chemistry.py`
Chemistry classifier: OxState failure	Compound dropped and logged	`tests/test_chemistry.py`
Split determinism	Same seed → identical split	`tests/test_splits.py`
Split stratification	Family proportions preserved within ±2%	`tests/test_splits.py`
Voronoi failure handling	Failed structures logged, dropped, fraction reported per family	`tests/test_featurizers.py`
Magpie output shape	$d_1 \approx 150$ for all non-failed structures	`tests/test_featurizers.py`

Criterion	Requirement	Validation
Feature caching	Deterministic: same input → same cached output	`tests/test_featurizers.py`

10.2 Conformal Prediction Correctness

Criterion	Requirement	Validation
Order-statistic convention	Implementation uses exact $R_{(\lceil (n+1)(1-\alpha) \rceil)}$ with `method="higher"`	`tests/test_conformal.py` (unit test on known residual vector with exact expected output)
Edge case: $k > n$	Interval is infinite (not clipped to $R_{(n)}$)	`tests/test_conformal.py`
Synthetic coverage (statistical check)	For Gaussian synthetic data with large n, empirical coverage at $\alpha = 0.10$ is within a binomial 95% confidence interval of the nominal level $1 - \alpha$	`tests/test_conformal.py`

10.2b Conformal Regression Checks

These are stability checks, not formal correctness conditions. They verify expected behavior on the actual dataset but may fluctuate with model fit, calibration size, and test size.

Check	Expected Behavior	Where Reported
Coverage on ID test (standard split)	Empirical coverage at $\alpha = 0.10$ approximately in $[0.89, 0.92]$	`scripts/run_evaluation.py`

10.3 SHAP Correctness

Criterion	Requirement	Validation
Efficiency axiom	$\sum_j \phi_j(\mathbf{x}) + \phi_0 = \hat{f}(\mathbf{x})$ for every test point (tolerance: 10^{-6})	`tests/test_integration.py`
SHAP runs on fixture	TreeSHAP completes on 100-crystal fixture without error	`tests/test_integration.py`

10.4 Model Training Correctness

Criterion	Requirement	Validation
MAE regression gate	MAE on frozen 100-crystal test fixture ≤ stored threshold	CI (`.github/workflows/ci.yml`)
End-to-end pipeline	100-crystal fixture → featurize → train → predict → evaluate, all steps pass	`tests/test_integration.py`

10.4b Training Stability Checks

These verify expected reproducibility behavior but depend on threading, hardware, and LightGBM internals. Not strict correctness conditions.

Check	Expected Behavior	Where Reported
Seed reproducibility	Same seed → same model predictions within floating-point tolerance on same hardware (requires deterministic LightGBM settings)	`tests/test_integration.py`

10.5 Phase B Correctness (ALIGNN)

Criterion	Requirement	Validation
Atom graph: neighbor selection	For each atom i, edges go to the $K_{\text{nn}} = 12$ nearest neighbors within cutoff $r_c = 8.0$ Å from `get_all_neighbors(8.0)`, sorted by distance	`tests/test_graph_builder.py`
Atom graph: edge directionality	For each selected neighbor pair (i, j), both directed edges (i, j) and (j, i) are present in the atom graph	`tests/test_graph_builder.py`
Atom graph: periodic images	Neighbors across periodic boundaries are correctly identified with the periodic distance	`tests/test_graph_builder.py`
Atom graph: edge feature dimension	Initial edge features are $K_e = 80$-dimensional Gaussian RBF expansion of distance (§7.2)	`tests/test_graph_builder.py`
Line graph: node count	Number of line-graph nodes equals number of atom-graph edges	`tests/test_graph_builder.py`
Line graph: triplet identification	For each pair of atom-graph edges (i, j) and (j, k) sharing atom j, there is a corresponding line-graph edge	`tests/test_graph_builder.py`
Line graph: angular feature	Each line-graph edge carries a 40-dimensional RBF expansion of $\cos\theta_{ijk}$ (§7.2, Eq. 7.2)	`tests/test_graph_builder.py`
Forward pass: output shape	ALIGNN returns a single scalar per crystal (batch dimension preserved)	`tests/test_graph_builder.py`
Forward pass: gradient flow	`loss.backward()` completes without error on a minimal batch	`tests/test_graph_builder.py`
Rotational invariance	Rotating input coordinates by a random $\mathrm{SO}(3)$ element leaves the predicted scalar unchanged (within floating-point tolerance 10^{-5})	`tests/test_graph_builder.py` (§7.3 sanity check)

11. Reported Findings

Phase A and Phase B are both complete. This section reports the measured findings against the theoretical predictions, including where the predictions were partially wrong. Numbers are drawn from the repository's `results/tables/` outputs and match the values reported in `README.md`.

11.1 Phase A Findings (Tiers 1–2)

Finding 1 — Composition baseline strength. *[Measured]* Magpie + LightGBM achieves MAE = 0.124 eV/atom on formation energy (standard split) and MAE = 0.514 eV on band gap. The formation-energy result confirms that composition alone explains most of the variance in DFT formation energies — consistent with the hypothesis in §2.1 that composition descriptors capture the dominant chemical-bonding energetics. The band gap result is substantially harder, also consistent with the §2.1 caveat that electronic-structure properties depend on geometry.

Finding 2 — Structure helps on standard splits, hurts under domain shift. *[Measured]* This is the finding that partially falsified the §2.2 prospective hypothesis. On the standard split, Tier 2 (Magpie + Voronoi) improves both targets: formation energy 0.124 → 0.105 eV/atom, band gap 0.514 → 0.440 eV. **But under domain shift, Tier 2 degrades:** band gap MAE rises from Tier 1's 0.537 eV to Tier 2's 0.593 eV on oxide-ID, and the degradation is worse on OOD families. Structural features learned on oxide geometries do not transfer to other chemistries, while composition features do. The original prospective hypothesis ("structure helps band gap more than formation energy") was correct on the standard split but missed the domain-shift reversal. This is an instance of a general principle: inductive biases that improve in-distribution performance can harm out-of-distribution transfer when the bias itself is distribution-specific.

Finding 3 — Domain-shift degradation pattern confirmed for formation energy. *[Measured]* Formation energy MAE ratios (Tier 1, OOD vs. ID oxide): sulfide 2.3×, nitride 4.8×, halide 6.3×. The hypothesized difficulty gradient (sulfide < nitride < halide, §5.2) is confirmed. Band gap degrades more moderately (1.1× / 1.4× / 2.4×) and the gradient is weaker, consistent with composition features transferring better for electronic properties.

Finding 4 — Conditional coverage collapses under shift. *[Measured]* The §4.5 theoretical prediction is confirmed with concrete numbers at $\alpha = 0.10$ (nominal 90% coverage):

Target	ID (oxide)	Sulfide	Nitride	Halide
Formation energy	90.2%	51.4%	24.7%	28.2%
Band gap	90.6%	89.3%	82.8%	60.0%

The marginal guarantee holds on ID oxides (within the $1/(n+1)$ bound derived in §4.4) and fails on OOD families, exactly as predicted by the exchangeability-violation argument in §4.5. A subtle additional result: **band gap coverage degrades less sharply than formation energy coverage.** The mechanism is that the band-gap model's residuals are heavier-tailed in-distribution, so $\hat{q}$ is already wide, and a wide interval absorbs more of the OOD distributional drift before coverage collapses. This is a teachable moment for practitioners: a model with *worse* ID accuracy can have *more robust* conformal coverage under shift, because its intervals were already conservative.

Finding 5 — Mixed training as domain randomization recovers most of the degradation asymmetrically. *[Measured]* Training on all chemistry families recovers 70–86% of formation-energy degradation and 46–58% of band-gap degradation. Halides benefit most for formation energy (86% recovery). **The**

asymmetry is interpretable in terms of concept shift: formation energy depends on chemical-bonding energetics that are approximately shared across families (mild concept shift), so mixing families in training acts as effective domain randomization. Band gap depends on electronic structure that differs more fundamentally across families (stronger concept shift), so pooling families yields smaller gains. The result is consistent with the §5.2 framing of the split as *approximated* (not exact) covariate shift.

Finding 6 — Calibration efficiency curve: the deployable UQ result. *[Measured, §4.7]* This is the finding that Phase A's theoretical §4.7 (per-family recalibration) was built to explain. Using n_{cal} OOD-family samples to recalibrate split conformal intervals restores coverage as follows (at $\alpha = 0.10$):

n_{cal}	Behavior
5	Unreliable — coverage std up to ±31% across families
10	>90% coverage for most families, still noisy
25	**95–99% coverage across all families and both targets** (slight overcoverage from small-n quantile conservatism)
50–100	Settles to nominal 90% coverage

The theoretical basis is the §4.7 per-family exchangeability argument: once we calibrate on a sample drawn from family k, the new calibration set is exchangeable with test points drawn from k, and the §4.4 marginal-coverage theorem applies *conditional on being in family* k. The empirical question was how small n_{cal} could be before quantile-estimation variance overwhelms the guarantee. **Answer:** $n_{\text{cal}} \approx 25$**.** The overcoverage at small n is the expected conservatism of the $\lceil (n+1)(1-\alpha) \rceil / n$ quantile index when n is tiny — as n grows, the index converges to the nominal $1-\alpha$ quantile.

Practical recommendation for deployment: A scientist encountering a new chemistry family needs ~25 DFT-validated samples before trusting the predictor's uncertainty estimates. This is the architecture-agnostic recipe the repo delivers.

Finding 7 — Tiers fail on nearly disjoint crystals (4% overlap). *[Measured]* The 50 worst-predicted crystals per tier share only 2/50 (4%) between Tier 1 and Tier 2. Tier 1 failures concentrate in oxides (43/50); Tier 2 failures distribute across families (18 oxide, 13 halide, 11 sulfide, 8 nitride). The interpretation is that composition-only and structure-aware models fail on different crystals for different reasons — composition features miss geometric outliers, structural features miss composition outliers. **This near-disjointness validates the §2 design choice** of building two distinct tiers rather than a single "best" feature set: the tiers are complementary, not redundant. An ensemble combining both could reduce worst-case errors substantially (a natural future-work direction).

11.2 Phase B Findings (Tier 3, ALIGNN)

Finding 8 — GNN halves formation-energy error. *[Measured]* ALIGNN (Tier 3) achieves MAE = 0.051 eV/atom on formation energy (standard split, single seed) — a 51% reduction vs. Tier 2's 0.105 and a 59% reduction vs. Tier 1's 0.124. R^2 improves from 0.946 (Tier 2) to 0.986. The result confirms the §7.1 hypothesis that **angular features via ALIGNN's line graph capture coordination geometry more effectively than hand-crafted Voronoi statistics.** The 51% error reduction exceeds the prospective expectation and is the strongest single-model result in the benchmark.

Finding 9 — GNN advantage is smaller for band gap. *[Measured]* Tier 3 band gap MAE = 0.337 eV vs. Tier 2's 0.440 — a 23% improvement. The smaller margin confirms that band gap prediction benefits

from structural information but is fundamentally harder than formation energy because electronic structure depends on orbital overlap, hybridization, and exchange-correlation effects that cannot be fully captured by geometric graph features alone. This is consistent with the §1.1 caveat that the PBE-GGA band gap is itself a systematically biased quantity, and with §2.1's observation that composition alone captures less of the band-gap variance than the formation-energy variance.

Finding 10 — GNN does not solve domain shift; sulfides reveal a systematic bias. *[Measured, surprising]* The §7.5 prospective hypothesis was that GNNs might generalize better under chemistry shift because of their more expressive representation. **This is partially falsified.** Under chemistry shift, Tier 3 formation energy degrades 15.7× on sulfides, 6.5× on nitrides, and 4.5× on halides. The sulfide degradation is *worse* than Tier 1's 2.3×. On nitrides and halides, ALIGNN actually beats Tier 1 OOD performance, so the failure is **sulfide-specific, not architectural**.

Diagnostic analysis of the sulfide failure: the prediction distribution is *shifted*, not *collapsed*. Measured on sulfide OOD: `y_true` mean = −1.120 eV/atom, `y_pred` mean = −0.223 eV/atom, giving a systematic bias of +0.897 eV/atom. The predicted variance is 76% of the true variance, so the model's outputs have reasonable spread — they are simply offset. **Interpretation:** ALIGNN has learned oxide-specific energy scales from the training distribution and cannot shift them for sulfides, despite the architecture having no explicit bias toward oxides. This is a form of implicit covariate-shift failure: the learned representation encodes distributional information that a linear readout cannot override at test time.

Family	Tier 1 MAE	Tier 3 MAE	Tier 3 / Tier 1	Systematic bias
Sulfide	0.280	0.918	3.3× worse	+0.897 eV
Nitride	0.578	0.379	1.5× better	+0.213 eV
Halide	0.754	0.262	2.9× better	−0.039 eV

Open question. Tier 3 uses a 3+3 layer configuration (DECISIONS.md topic 22, chosen for training cost). Whether the sulfide bias would differ at the published 4+4 depth is an open question — not resolved by this benchmark. A practitioner deploying ALIGNN-family models on a new chemistry class should verify prediction distributions against a small DFT-validated calibration set before trusting outputs. The recipe from Finding 6 applies.

Finding 11 — The UQ finding generalizes to GNNs. *[Measured]* Conformal prediction intervals calibrated on oxides collapse on OOD families for Tier 3, just as they do for Tiers 1–2. The §4.7 per-family recalibration recipe (~25 OOD samples restore coverage) applies regardless of model architecture. This is expected from the theory: split conformal prediction is model-agnostic, so the exchangeability-violation argument in §4.5 and the per-family recovery argument in §4.7 apply identically to any predictor. **The calibration sweep from Finding 6 is therefore the repo's most architecture-agnostic, most deployable result.**

11.3 Summary: Theory vs. Measurement Scorecard

#	Prospective hypothesis	Result
1	Magpie MAE < 0.15 eV/atom on formation energy	✓ Confirmed (0.124)
2	Structure helps band gap more than formation energy	!! Confirmed on standard split, *reversed* under domain shift

#	Prospective hypothesis	Result
3	Difficulty gradient sulfide < nitride < halide	✓ Confirmed for formation energy, weaker for band gap
4	Conditional coverage breaks under shift	✓ Confirmed; magnitude measured
5	Mixed training helps	✓ Confirmed; asymmetric across targets (interpretable)
6	Per-family recalibration restores coverage	✓ Confirmed; $n_{\text{cal}} \approx 25$ is the threshold
7	Tiers fail on disjoint crystals	✓ Confirmed (4% overlap)
8	GNN outperforms tabular on standard split	✓ Confirmed (51% MAE reduction on formation energy)
9	GNN improves band gap less than formation energy	✓ Confirmed (23% vs. 51% improvement)
10	GNN generalizes better under shift	× *Partially falsified* — sulfide-specific failure
11	Per-family recalibration works for GNNs	✓ Confirmed (architecture-agnostic)

Two results disagreed with the original theory: Finding 2 (structure as a double-edged sword) and Finding 10 (GNN does not solve domain shift, sulfide-specific systematic bias). Both are genuine science — they reveal that inductive biases improving in-distribution performance can harm OOD transfer, and that the architecture-vs-shift question is not settled by "use a bigger model." These negative results are the most intellectually honest material in the repo and are framed that way in the README.

---

12. Notation Reference

Symbol	Meaning	Units / Range
E_f	Formation energy per atom	eV/atom
E_g	Band gap	eV, ≥ 0
N_{atoms}	Number of atoms in unit cell	integer
$\mathbf{x}$	Feature vector (Tier 1: $d_1 \approx 150$; Tier 2: $d_2 \approx 200$)	$\mathbb{R}^d$
y	Target property (formation energy per atom or band gap)	$\mathbb{R}$
$\hat{f}(\mathbf{x})$	Model prediction	same units as y
α	Conformal miscoverage level	$(0, 1)$, typically 0.10
$\hat{q}$	Conformal quantile (half-width of prediction interval)	same units as y
R_i	Nonconformity score (absolute residual)	same units as y
$R_{(k)}$	k-th order statistic of sorted residuals	same units as y
$C(\mathbf{x})$	Conformal prediction interval	$[\hat{y} - \hat{q}, \hat{y} + \hat{q}]$
Δ_k	Domain-shift degradation ratio for family k	dimensionless, > 0
$\phi_j(\mathbf{x})$	SHAP value of feature j for input $\mathbf{x}$ (tree-path-dependent)	same units as y
$I_j^{(k)}$	Global importance of feature j in family k	same units as y

Symbol	Meaning	Units / Range
τ	Chemistry-family purity threshold	dimensionless, $= 0.80$
$\mathcal{F}_k$	Chemistry family k	$\{$oxide, sulfide, nitride, halide$\}$
$\mathcal{H}_k$	Anion species set for family k	e.g., $\{\mathrm{O}\}$, $\{\mathrm{F}, \mathrm{Cl}, \mathrm{Br}, \mathrm{I}\}$
$\phi_k(s)$	Family fraction: proportion of anion sites in family k's species set	$[0, 1]$
n	Calibration set size	integer
n_{cal}	Per-family recalibration set size (§4.7)	integer, $\in \{5, 10, 25, 50, 100\}$ in the sweep
$\hat{q}^{(k)}$	Per-family conformal quantile (§4.7)	same units as y
$C^{(k)}(\mathbf{x})$	Per-family prediction interval (§4.7)	$[\hat{y} - \hat{q}^{(k)}, \hat{y} + \hat{q}^{(k)}]$
M	Number of LightGBM boosting rounds	integer, ≤ 1000
ν	Learning rate (shrinkage) for gradient boosting	0.05
L_{train}	Training loss (squared error for LightGBM; L2 + weight decay for ALIGNN)	—
MAE	Mean absolute error (primary evaluation metric)	same units as y

ALIGNN-specific notation (§7):

Symbol	Meaning	Units / Range
$\mathcal{G}_a = (\mathcal{V}_a, \mathcal{E}_a)$	Atom graph	directed multigraph
$\mathcal{L}(\mathcal{G}_a) = (\mathcal{V}_\ell, \mathcal{E}_\ell)$	Line graph (nodes are atom-graph edges; edges are bond-angle triplets)	directed graph
Z_i	Atomic number of atom i	$\{1, \dots, 118\}$ in the raw data; element vocabulary for the embedding is inherited from the `alignn` package's atomic feature table
r_c	Neighbor cutoff for atom-graph edge construction	8.0 Å
K_{nn}	Nearest-neighbor cap per atom (DECISIONS.md topic 18)	12
d_{ij}	Interatomic distance (periodic-image aware)	Å
K_e	Number of Gaussian radial basis centers for edge distance expansion (§7.2 Eq. 7.1)	80
$\mathbf{e}_{ij}^{(l)}$	Atom-graph edge feature vector at ALIGNN layer l	$\mathbb{R}^{K_e} = \mathbb{R}^{80}$
$\mathbf{v}_i^{(l)}$	Atom-graph node feature vector at ALIGNN layer l	$\mathbb{R}^{64}$ (ALIGNN default hidden size)
θ_{ijk}	Bond angle at shared atom j between bonds (i, j) and (j, k)	radians, $[0, \pi]$
K_θ	Number of RBF centers for triplet angular feature expansion (§7.2 Eq. 7.2)	40
$\mathbf{t}_{ijk}$	Line-graph edge feature vector (RBF expansion of $\cos\theta_{ijk}$)	$\mathbb{R}^{K_\theta} = \mathbb{R}^{40}$

Symbol	Meaning	Units / Range
L	Number of ALIGNN layers (ALIGNN + GCN)	3+3 in this project (DECISIONS.md topic 22), 4+4 in the published default
$\mathbf{u}$	Crystal-level feature vector after global mean pooling (§7.5)	$\mathbb{R}^{64}$
λ	Weight decay coefficient for ALIGNN training	10^{-5}
$\mathrm{CN}_i^{\mathrm{eff}}$	Effective Voronoi coordination number of atom i (used in Tier 2 tabular features, not ALIGNN)	$\mathbb{R}_{>0}$